Laboratorul 0: Aplicații introductive

Capitole utile din Datasheet ATmega324

- 1. Pin Configurations - pag. 2
- 14. I/O-Ports - pag. 72

0. Structura laboratorului

- Planificare și orar
- Reguli generale și de notare

1. Ce este un microcontroller?

Un microcontroller(µC) este o componentă electronică care integrează un microprocesor şi dispozitive periferice, punându-se accent pe un cost de producție mic și un consum energetic redus, altfel spus pe optimizarea aplicației. Principala diferenţă dintre un microcontroller (µC) şi un microprocesor (µP) o constituie faptul că un µC integrează atât unitatea de procesare cât și memorie de program, memorie de date, interfeţe de intrare-ieşire și periferice.

Un µC operează la frecvenţe reduse, în general la zeci sau sute de MHz. Cu toate acestea, microcontroller-ele sunt utilizate într-o gamă largă de aplicaţii în diverse domenii. Dispozitivele unde µC sunt utilizate pot fi produse de uz general (mașini de spălat, frigidere, automate de cafea), sisteme industriale dedicate (relee de protecție, controllere industriale, sisteme de tip interfață om-mașină - HMI), sisteme din industria aerospaţială și multe altele.

În funcție de tipul aplicațiilor pentru care sunt dedicate acestea, majoritatea µC nu au o magistrală externă de adrese sau date deoarece toate memoriile utilizate (volatile și nevolatile) sunt interne, ducând la integrarea acestora în capsule cu un număr mic de pini şi reduse ca dimensiuni, ceea ce reduce costurile de producţie și consumul energetic. Există desigur și microcontrollere care furnizează magistrale externe pentru date și adrese (multiplexate în timp sau magistrale dedicate) în scopul interfațării memoriilor externe (RAM, FLASH, ROM).

Cele mai întâlnite structuri din circuitul integrat al unui µC sunt următoarele:

- Unitatea centrală de procesare (*µP core*) cu o arhitectură care poate fi pe 8, 16, 32 sau 64 de biţi.
- Memorie de date volatilă (RAM) și/sau non-volatilă (Flash sau EEPROM)
- Memorie de program non-volatilă (Flash sau EEPROM)
- Porturi digitale de intrare-ieşire de uz general (GPIO - General Purpose Input Output)
- Interfeţe seriale de comunicație (USART, SPI, I2C, PCM, USB, SDIO etc.)
- Interfețe ethernet
- Interfețe pentru afișaj grafic (LVDS, HDMI sau alte protocoale dedicate controlului afișajelor LCD)
- Timere (cu rol intern, sau utilizate în generarea semnalelor periodice -ex: PWM-, sau ca watchdog)
- Convertoare analog-digitale și digital-analogice, front-end-uri analogice și alte circuite dedicate semnalelor analogice
- Sursă de tensiune integrată
- Interfață pentru programare şi debugging

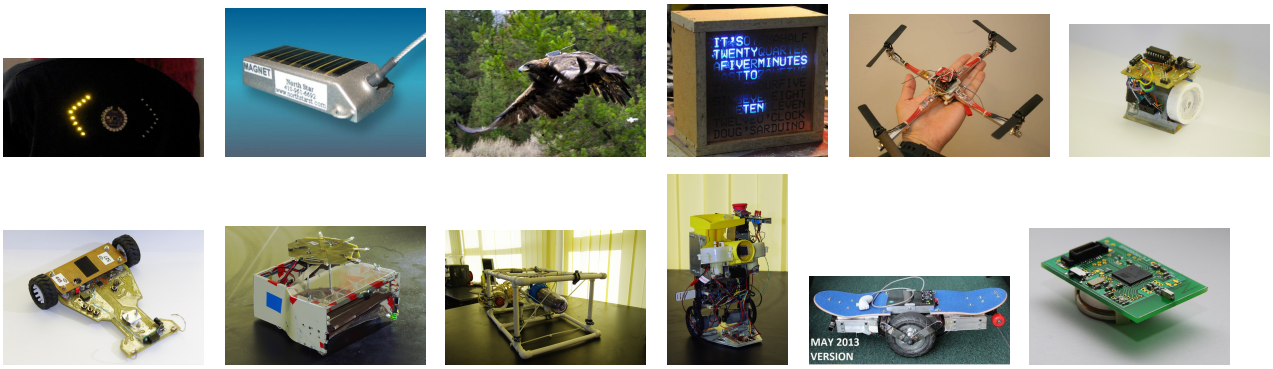
Un **periferic** reprezintă orice dispozitiv, intern sau extern, care se conectează la un sistem de calcul și îi extinde funcționalitatea de bază. În cazul microcontroller-ului, există o serie de astfel de periferice incluse direct în circuitul integrat (cele menționate mai sus). Deși nu seamănă cu perifericele unui PC (monitor, placă grafică, imprimantă, tastatură, mouse, etc.), fără ele microcontroller-ul nu ar putea interacționa cu mediul exterior. Mai mult, perifericele ne ajută să conectăm alte elemente mai performante la controller si

sa îi putem oferi funcționalități similare unui sistem PC (conexiune la internet, linie de date USB, display grafic etc.)

Costul unui µC depinde în mare măsură de numărul de periferice integrate, frecvența de lucru a procesorului, cantitatea de memorie și alți parametri amintiți mai sus. Cu cât caracteristicile acestuia sunt mai avansate, cu atât crește prețul de producție. Arhitecturile de µC pe piaţă la ora actuală variază în limite largi, de la chip-uri cu doar 2 pini de I/O până la procesoare digitale de semnal (DSP) sau procesoare cu arhitecturi avansate pe 32 sau chiar 64 de biţi (ex: ARM, PIC32, STM32 sau AVR32).

În concluzie, un microcontroller este un circuit integrat ce reunește o unitate de procesare (CPU), memorii(volatile RAM, nevolatile EEPROM, Flash, ROM) și diverse periferice ce îi permite acestuia să comunice cu mediul extern. Microcontrollerele sunt utilizate în diverse aplicații, din diverse domenii, de la aplicații casnice și industriale până la aviație și sateliți. Există o gamă largă de microcontrollere disponibile și acestea se aleg în funcție de aplicație, având în vedere mai ales optimizarea costului și a consumului energetic pentru dispozitivul unde µC urmează a fi folosit.

Ce pot face cu un microcontroller?



- [1] - Turn signal biking jacket [http://www.instructables.com/id/turn-signal-biking-jacket/]
- [2, 3] - Bird tracking [https://www.northstarst.com/]
- [4] - The word clock [http://www.instructables.com/id/The-Word-Clock-Arduino-version/]
- [5] - Palm size quad-copter [http://www.rcgroups.com/forums/showthread.php?t=1335765]
- [6] - Robot microsumo (RoboWarrior)
- [7] - Robot line follower (Tare N' Bară)
- [8] - Robot ce joacă fotbal (IronFoot)
- [9] - Robot submarin (R0X1)
- [10] - Robot vânător de mamuți (Many the mammoth hunter)
- [11] - Self-balancing skateboard [http://www.instructables.com/id/Easy-build-self-balancing-skateboardrobotsegway-/]
- [12] - Nod senzorial (Sparrow)

Pentru alte exemple, puteți începe căutarea de aici: https://www.hackster.io/projects/tags/microcontroller [https://www.hackster.io/projects/tags/microcontroller]

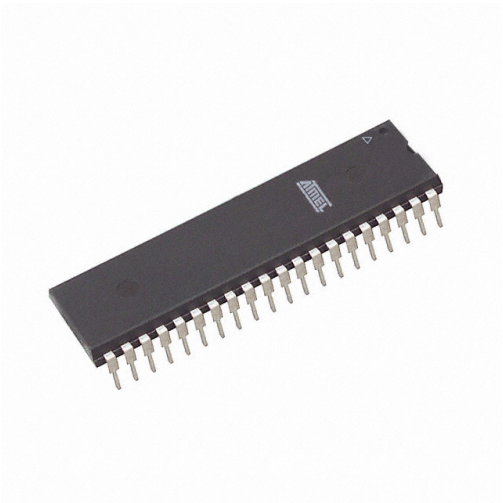
2. Microchip (Atmel) AVR

Famila AVR de la Microchip este formată din microcontrollere cu arhitectură Harvard pe 8 biţi şi set redus de instrucţiuni (RISC).

Memoriile Flash, EEPROM, şi SRAM sunt integrate în acelaşi chip, înlăturând nevoia de memorie externă. Programul este format din instrucţiuni de 16 biţi lungime care sunt stocate în memoria Flash non-volatilă. Mărimea memoriei de program este indicată de numele componentei respective. De exemplu, ATmega128 are 128kB de memorie Flash. Spaţiul de adresa este format din registrele generale, registrele de I/O şi memoria SRAM. Sunt în total 32 de registre generale a câte 8 biţi.

AVR au o unitate de execuţie în bandă de asamblare cu două niveluri, acest lucru permiţând că următoarea instrucţiune să fie adusă din memorie (fetch) în timp ce instrucţiunea curentă este în execuţie. Majoritatea instrucţiunilor se execută într-un singur ciclu de instrucţiune, acest lucru permiţând atingerea unui throughput de 1MIPS pe MHz.

2.1. ATmega324



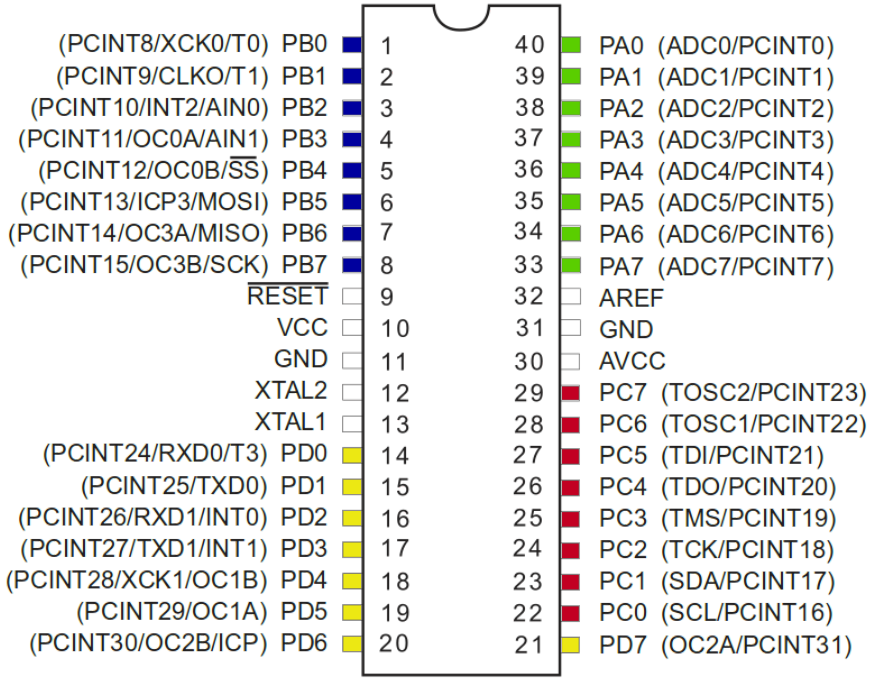
În cadrul laboratoarelor și a proiectului de PM, veți utiliza ATmega324, un microcontroller pe 8 biți din familia megaAVR. Registrele şi magistrala internă de date sunt pe 8 biţi.

Caracteristici generale

- 32 KB Flash (determină dimensiunea maximă a programului care poate fi executat)
- 1 KB EEPROM
- 2 KB RAM
- 20 MHz frecvența maximă de lucru
- Tensiune de alimentare între 1.8V și 5.5 V
- 6 canale de PWM
- 8 canale de ADC, cu rezoluție de 10 biți
- 4 porturi digitale de I/O (GPIO - General Purpose I/O) , fiecare cu 8 pini, în total 32 de pini de I/O
- 3 timere (două pe 8 biți și unul pe 16 biți)
- Interfeţe de comunicație seriale: USART, SPI, TWI
- Interfaţe de programare ISP și debug JTAG

Perifericele interne chip-ului pot fi accesate din exterior prin intermediul pinilor. Capsula microcontroller-ului are 40 de pini (prezentați mai jos), dintre care 8 sunt pentru alimentare sau funcții auxiliare, iar 32 sunt pentru I/O. GPIO-urile sunt împărțite în patru porturi (A, B, C și D) a câte 8 pini fiecare.

Pentru mai multe detalii, consultați datasheet-ul (documentația tehnică sumarizată) a microcontrollerului.

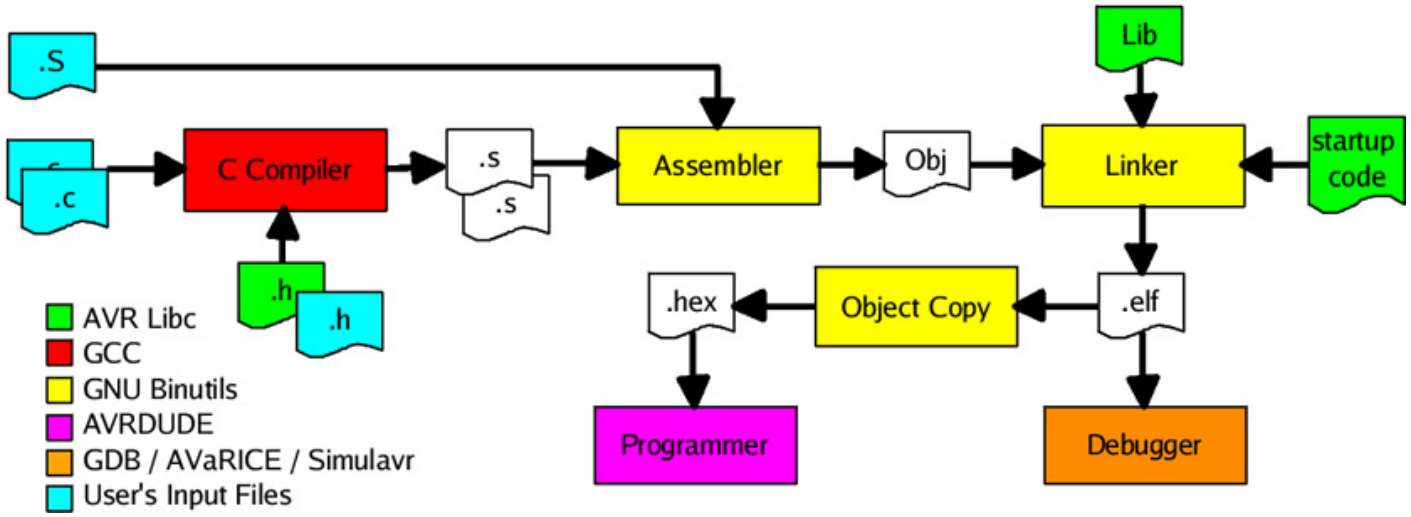


3. Dezvoltarea programelor pentru AVR

3.1. Dezvoltare, compilare

Pentru dezvoltarea programelor putem folosi orice editor de text (vim [https://www.vim.org/], emacs [https://www.gnu.org/software/emacs/], Notepad++ [https://notepad-plus-plus.org/], Programmer's Notepad [http://www.pnotepad.org/], etc). Putem utiliza de asemenea şi IDE-ul Atmel Studio [https://www.microchip.com/mplab/avr-support/atmel-studio-7], oferit de Microchip.

În cadrul laboratoarelor vom scrie cod C, nu şi assembly (AVRASM), compilatorul fiind avr-gcc [https://gcc.gnu.org/wiki/avr-gcc]. Acesta poate fi folosit atât pe sisteme Linux/Unix cât şi pe sisteme Windows (WinAVR [http://winavr.sourceforge.net/]). WinAVR include şi biblioteca de C, avr-libc, pe sistemele Linux/Unix ea trebuie instalată separat.

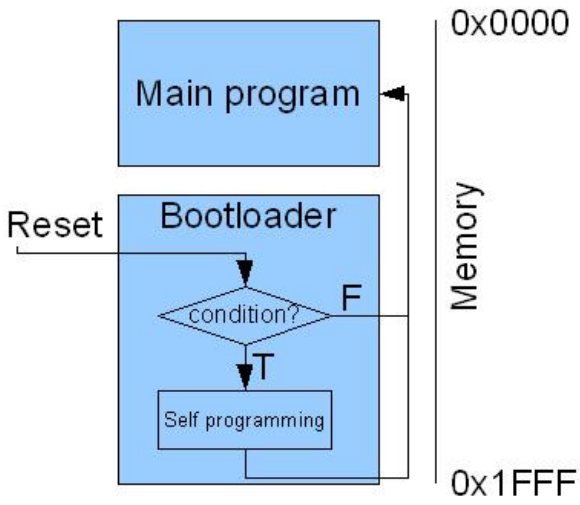


Deși microcontroller-ul nostru este pe 8 biți, în codul C putem folosi variabile întregi pe 16/32/64 de biţi şi chiar și în virgulă mobilă. Compilatorul este cel care se va ocupa de translatarea instrucţiunilor în cod asamblare ce lucrează pe 8 biţi.

3.2. Programarea microcontroller-ului

Pentru programarea unui microcontroller putem folosi diverse metode, cum ar fi programarea pe interfaţă serială, folosirea unui programator ISP (In-System Programming) ce foloseşte interfaţa serială SPI sau folosirea unui bootloader.

În cadrul acestui laborator vom folosi ultima variantă, un **bootloader** fiind un program încărcat (folosind, spre exemplu, un programator ISP) la sfârşitul memoriei de program a microcontroller-ului. Execuţia codului încărcat astfel va începe din zona de boot. Dezavantajul acestei metode este spaţiul ocupat de bootloader în memoria de program.



Folosirea bootloader-ului presupune îndeplinirea unei condiții la pornirea microcontroller-ului. Pe plăcile de laborator acesta este rulat la fiecare RESET, însă putem pune orice condiție pe care o poate deduce procesorul (spre exemplu, dacă un buton este apăsat sau nu). Unele microcontrollere au o memorie ROM dedicată bootloader-ului, cu care acestea sunt prevăzute din fabrică (ex: STM32).

Pentru a încărca programul pe controller trebuie să folosim un utilitar de pe calculator. Un exemplu de folosire a utilitarului **avrdude** pe Linux si pe Windows găsiți mai jos.

```
$ avrdude -c arduino -P /dev/ttyUSB0 -b 57600 -p atmega324p -U flash:w:lab0.hex:a

C:/> avrdude -c arduino -P COM12 -b 57600 -p atmega324p -U flash:w:lab0.hex:a
```

4. Prima aplicație

4.1. Legătura cu mediul exterior: traductoare, senzori și actuatori

Pentru a putea intefața cu mediul exterior, sunt utilizate diferite componente electronice care au rol fie de actuator (modifică starea mediului exterior) sau de traductor/senzor (sunt influențate de mediul exterior și oferă informații microcontroller-ului despre diverși parametri).

Actuatorii sunt acele elemente care influențează mediul exterior. Putem spune că actuatorii sunt elementele ce pot fi comandate și care pot interacționadirect cu mediul pentru a influența diferiți parametri (ventilatoare, indicatoare sonore - buzzere , indicatoare luminoase, rezistențe de încălzire, etc). **Uneori pentru a putea activa un actuator, este nevoie de un element de acționare.** De exemplu, **pentru a putea porni un motor(actuator) prin comanda dată de microcontroller, trebuie să folosim un element de acționare** (care pornește ventilatorul) intermediar care să poată comanda curenți mari (prin ventilator) folosind curenți foarte mici de comandă (ieșirea microcontrollerului). În acest exemplu, soluția ar fi utilizarea unui tranzistor.

Traductoarele (sau traductorii) sunt acele elemente care își modifică proprietățile electrice în funcție de parametrii mediului exterior și oferă microcontroller-ului informații despre mediul exterior. De exemplu: un fotorezistor va avea rezistența electrică mult mai mare în lipsa luminii, iar când aceasta este iluminată va avea o rezistență electrică de zeci sau chiar sute de ori mai mică. Un alt exemplu de traductor este butonul

simplu normal deschis: când este apăsat el închide un contact, când nu este apăsat, contactul din interior este deschis. În funcție de tipul traductoarelor, acestea pot avea nevoie de prelucrarea semnalului înainte ca acesta să fie preluat de microcontroller (signal conditioning) - de exemplu fotorezistorul trebuie folosit într-un montaj cu divizor de tensiune sau cu sursă de curent -, sau pot fi conectate direct la microcontroller - de exemplu butonul.

În cadrul acestui laborator veți utiliza LED-uri pe post de indicator luminos de semnalizare a stării unui pin și butoane pentru a transmite informație către microcontroller.

Diode LED

LED-urile - Light Emitting Diode - numite și diode electroluminescente emit lumină când ele sunt polarizate direct. În funcție de semiconductorii care sunt utilizați în construcție, și de plasticul ce are rol de lentilă de dispersie și filtru optic, LED-urile pot emite lumină în diferite culori. **Nu confundați LED-urile cu becurile!** În timp ce becurile emit lumina prin încalzirea unui filament de tungsten la temperaturi foarte mari, LED-urile emit lumina trecând un curent printr-un material semiconductor (joncțiune p-n). Spre deosebire de becuri, al căror randament este de sub 5% - majoritatea energiei primite de ele se pierde prin efect caloric -, LED-urile au un randament de transformare a energiei electrice în energie luminoasă mult mai mare.

LED-urile pot fi utilizate pe post de indicator luminos (adesea utilizate în diferite aparate pentru a semnaliza faptul că aparatul este pornit și realizează un anumit lucru), sau pentru iluminare, caz în care sunt utilizate LED-uri de putere.

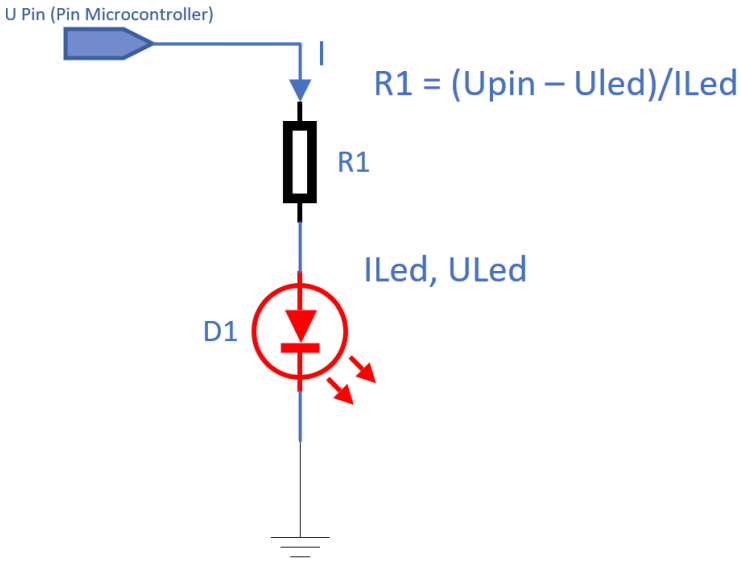
În cadrul laboratorului LED-urile sunt utilizate pentru a indica starea unui pin.

LED-urile sunt diode, așadar, curentul prin acestea crește exponențial cu creșterea tensiunii aplicate. Pentru a utiliza un LED pentru indicarea stării unui pin (mai degrabă spus pentru a indica prezența de tensiune), curentul prin LED trebuie limitat. Aceasta se poate realiza în cel mai simplu mod prin înserierea unei rezistențe cu LED-ul.

Calculul rezistenței de limitare a curentului:

Un LED este proiectat să opereze la un curent nominal (ex: 10mA). Căderea de tensiune pe LED urile indicatoare, de mică putere, la acest curent, este dată de culoarea LED-ului, puterea acestuia și de temperatură. Ea poate varia de la 1.7V în cazul LED-urilor roșii, 2-2.3V pentru cele Orange, galbene și verzi și ajunge la 3.2-3.5V în cazul LED-urilor albastre și albe.

Schema utilizată este următoarea:



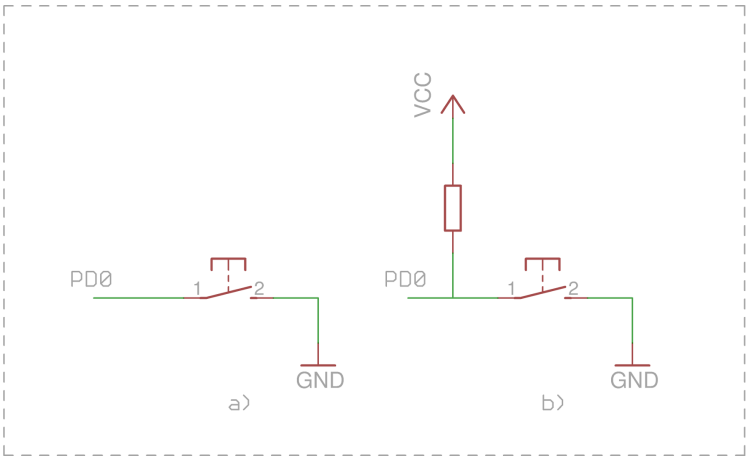
Schema de lucru și calculul rezistenței de limitare a curentului prin LED

Întrucât microcontroller-ul are ieșirile comandate folosind stâlp Push-Pull CMOS, tensiunea Upin în starea de 1 logic va fi apropiată de tensiunea de alimentare a microcontroller-ului.

Exemplu:Dacă alimentarea microcontroller-ului este de 5V pentru un LED roșu ce dorim să îl folosim la 10mA, specificat de producător cu o cădere de tensiune de 1.7V, trebuie să folosim o rezistență de 330 de ohmi.

4.1.2. Butoanele

Cel mai simplu mod de interacțiune al utilizatorului cu un microcontroller îl constituie folosirea butoanelor. Modul de conectare al unui push-button este dat în figura de mai jos:



Conectarea unui push-button: a) incorect, cu intrare flotantă, b) corect, cu rezistență de pull-up

a) arată un buton conectat la pinul PD0 al µC. La apăsarea butonului, intrarea PD0 va fi legată la GND, deci va fi în starea logică "0". Acest mod de legare este incorect deoarece atunci când butonul nu este apăsat intrarea se află într-o stare nedefinită (ca și cum ar fi lăsată în aer), ea nefiind conectată nici la GND, nici la Vcc. Această stare se numește stare de impedanță mărită și nu poate fi citită de către circuitele interne ale µC deoarece un bit dintr-un registru poate să ia doar valorile 0 sau 1. În practică, citirea unei intrări în stare de impedanță mărită va produce un rezultat de 1 sau 0 în funcție de condițiile de mediu. Spre exemplu, dacă apropiem degetul de acea intrare, citirea va fi 1, iar, daca îndepărtăm degetul, citirea va fi 0.

b) arată modul corect de conectare al butonului, folosind o **rezistență de pull-up** între pinul de intrare și Vcc. Această rezistență are rolul de a aduce intrarea în starea "1" logic atunci când butonul este liber prin "ridicarea" potențialului liniei la Vcc. Alternativ, se poate folosi o **rezistență de pull-down** (conectată la GND), caz în care intrarea este ținută in starea logică "0" cât timpul butonul nu este apăsat.

Pentru a economisi spațiu exterior, în µC ATmega324, aceste rezistențe au fost incluse în interiorul circuitului integrat. Inițial ele sunt dezactivate iar activarea acestora se poate face prin software scriind o valoare în registrul de ieșire (`PORTn`) al unui port care a fost configurat drept intrare. Bitul `PUD` din registrul `SFIOR` este setat inițial pe 0, adică atunci când se scrie în `PORTn` se activează rezistențele de pull-up.

4.1.3. Avertizorul/indicatorul sonor

Un indicator sonor de tip buzzer/speakereste un dispozitiv electronic, capabil să producă sunet. Acesta poate fi mecanic, electromecanic sau piezoelectric. Se folosesc de regulă în sisteme de alarmă, ceasuri sau ca feedback sonor pentru anumite acțiuni.

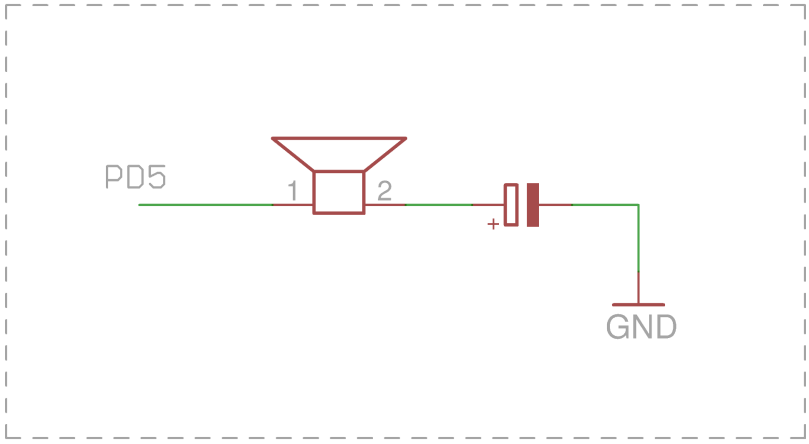
Buzzerele sunt indicatoare sonore care folosesc semnale electric periodic pentru a genera sunetul. Buzzerele pasive sunt buzzerele care trebuie alimentate folosind impulsuri/semnal variabil. Există și buzzere, denumite active, care conțin circuitul oscilator, și pot fi alimentate în curent continuu. Acestea genereaza un sunet cât timp primesc alimentare.

Buzzerul utilizat în cadrul laboratorului este un buzzer pasiv, care este cuplat în alternativ prin înserierea cu o capacitate. Acest buzzer, pentru a funcționa trebuie să primească un semnal electric variabil, sunetul fiind emis în frecvența semnalului cu care acesta este atacat.

Pentru a genera un astfel de semnal trebuie urmați următorii pași:

- setăm pinul ca fiind pin de ieșire
- setăm pinul pe valoarea high
- așteptăm jumătate din perioadă
- setăm pinul pe valoarea low
- așteptăm cealaltă jumătate din perioadă.

În funcție de durata perioadei, putem folosi macro-urile definite în biblioteca util/delay.h: _delay_ms sau _delay_us.



Connectarea unui speaker

4.2. Lucrul cu registrele I/O

Microntroller-ul ATmega324 oferă 4 porturi I/O a câte 8 pini, iar intern, fiecare port are asociat trei registre a câte 8 biți prin care utilizatorul poate controla *la nivel de pin* fluxul datelor: poate scrie/citi date în/din portul respectiv. Aceste trei registre sunt:

- **DDRn** - *Data Direction Register*
  - stabilește direcţia pinilor portului
  - dacă bitul *x* are valoarea **0** atunci pinul *x* este de **intrare**
  - dacă bitul *x* are valoarea **1** atunci pinul *x* este de **ieșire**
- **PORTn** - *Data Register*
  - stabilește valorile de ieşire ale pinilor sau activează/dezactivează rezistenţele de pull-up
  - dacă bitul *x* are valoarea **0** atunci
    - dacă pinul *x* este de ieșire el va avea valoarea **LOW**
    - daca pinul *x* este de intrare rezistența de pull-up va fi dezactivată
  - data bitul *x* are valoarea **1** atunci
    - daca pinul *x* este de ieșire el va avea valoarea **HIGH**
    - dacă pinul *x* este de intrare rezistența de pull-up va fi activată
- **PINn** - *Input Pins Address*
  - putem citi date de pe portul respectiv
  - dacă pinul *x* are valoarea **LOW** atunci bitul *x* va avea valoarea **0**
  - daca pinul *x* are valoarea **HIGH** atunci bitul *x* va avea valoarea **1**

n poate să fie A, B, C sau D în funcţie de portul selectat. *x* poate să fie între 0 și 7.

Descrierea detaliată a porturilor şi registrelor corespunzătoare acestora se găseşte în datasheet-ul ATmega324, în capitolul *I/O Ports*.

Registrele DDRn, PORTn, PINn, precum şi registrele perifericelor nu fac parte din cele 32 de registre de uz general. Ele nu sunt folosite pentru stocarea datelor, ci au ca rol comunicarea cu perifericul corespunzător.

Pentru a face accesul la acestea mai simplu, ele sunt **mapate în memorie**. Astfel, pentru a scrie o valoare într-un registru este necesar să se scrie valoarea la adresa de memorie corespunzătoare registrului. Adresele lor se pot vedea în datasheet, în capitolul *Register Summary*, iar în avr-libc sunt definite macro-uri pentru fiecare dintre aceste adrese.

Pentru a scrie o valoare într-un registru se poate face direct atribuirea unei valori, însă acest mod nu face codul foarte ușor de înteles. *Recomandarea* noastră este de a folosi măștile pe biți (`(1 << x)`) sau macro-ul `_BV(x)`. De asemenea recomandăm folosirea numelui pinului în loc de indexul acestuia, pentru că numele acestora sunt sugestive pentru funcția pe care o îndeplinesc (spre exemplu, în registrul ADCSRA al convertorului analog-digital, bitul ADEN este bitul de enable). Numele pinilor, ca și în cazul registrelor, sunt deja definite în avr-libc.

Operație	Formă
Scriere bit pe 1	`register |= (1 << bit_index)`
Scriere bit pe 0	`register &= ~(1 << bit_index)`
Toggle bit	`register ^= (1 << bit_index)`
Citire bit	`register & (1 << bit_index)`

Lucrul cu registre

```
DDRB = 8;               // AȘA NU.
DDRB = (1 << 3);        // Nici așa, este hardcodat indexul pinului.
DDRB |= (1 << PB3);     // AȘA DA.
DDRB |= _BV(PB3);       // AȘA DA.
```

Detalii despre lucru cu biții din registre puteți citi și la secțiunea tutoriale de pe wiki [https://ocw.cs.pub.ro/courses/pm/tutorial/biti].

4.2.1. Exemplu de lucru cu ieșiri

Să presupunem că avem un LED legat la pinul 1 al portului B (numit PORTB1 sau PB1). Pentru a aprinde sau stinge LED-ul trebuie să urmăm următorii pași:

- Pinul PB1 trebuie configurat ca ieșire
  - Bitul 1 (PB1) din registrul DDRB va fi 1
- Pentru a aprinde LED-ul trebuie ca pinul PB1 sa ia valoarea HIGH
  - Bitul 1 (PB1) din registrul PORTB va fi 1
- Pentru a stinge LED-ul trebuie ca pinul PB1 sa ia valoarea LOW
  - Bitul 1 (PB1) din registrul PORTB va fi 0

4.2.2. Exemplu de lucru cu intrări

Să presupunem ca avem un buton legat la pinul 4 al portului D (numit PORTD4 sau PD4), din cazul b) prezentat anterior. Pentru a determina starea butonului (apăsat sau liber) trebuie să urmăm următorii pași:

- Pinul PD4 trebuie configurat ca intrare
  - Bitul 4 (PD4) din registrul DDRD va fi 0

- Pentru a determina starea de apăsare a butonului trebuie să citim valoarea pinului la care este atașat. Acesta va fi 1 atunci când butonul este liber și 0 atunci când butonul este apăsat
  - Citim valoarea bitului 4 (PD4) din registrul PIND

4.3. Hello World

Vom scrie un program care stinge și aprinde un LED la intervale de 500 ms. Acest lucru se face modificând tensiunea unuia dintre pinii microcontroller-ului, în cazul nostru pinul 0 din portul C (PC0).

lab0.c

```
#include <avr/io.h>
#include <util/delay.h>

int main() {
        /* Setăm pinul 0 al portului C ca pin de ieșire. */
        DDRC |= (1 << PC0);

        while(1) {
                /* Inversăm starea pinului. */
                PORTC ^= (1 << PC0);

                _delay_ms(500);
        }

        return 0;
}
```

Pentru compilare folosim următorul Makefile:

Makefile

```
# Linux
PORT ?= /dev/ttyUSB0
# Windows
# PORT ?= COM1

all: lab0.hex

lab0.hex: lab0.elf
        avr-objcopy  -j .text -j .data -O ihex $^ $@
        avr-size lab0.elf

lab0.elf: lab0.c
        avr-g++ -mmcu=atmega324p -DF_CPU=12000000 -Os -Wall -o $@ $^

upload: lab0.hex
        avrdude -c arduino -P $(PORT) -b 57600 -p atmega324p -U flash:w:$<:a

clean:
        rm -rf lab0.elf lab0.hex
```

Compilatorul folosit este **avr-gcc**. Flag-urile au următoarea semnificație:

- -mmcu: Informează compilatorul despre tipul microcontroller-ului pentru care trebuie să genereze codul
- -DF_CPU=12000000: Definește macro-ul F_CPU care indică frecvența de lucru la microcontroller-ului
- -Os: Optimizează programul în privința memoriei ocupate (destul de folositor având în vedere memoria limitată prezentă pe chip)
- -Wall: Activează toate mesajele de avertisment

Fișierul care poate fi programat pe plăcuță trebuie să fie în formatul *ihex*. De aceea este necesară extragerea secțiunilor de date și cod din fișierul elf obținut în urma compilării integrarea lor într-un fișier *ihex*. Acest pas folosește utilitarul **avr-objcopy**.

Comanda **avr-size** arată câtă memorie ocupă diverse secțiuni de cod ale programului. Trebuie avut în vedere că în memoria SRAM de date, în afară de secțiunea .data, o să fie pusă și stiva. Pentru a putea rula programul pe microcontroller-ul ATmega324 trebuie că secțiunea .text să fie <32 KB și secțiunea .data să fie <2 KB.

Rezultatul compilării este fișierul **lab0.hex**. Acesta urmează a fi încărcat în microcontroller folosind regula **upload**.

5. Exerciții

Capitole utile din Datasheet ATmega324

- 1. Pin Configurations - pag. 2
- 14. I/O-Ports - pag. 72

Task 0 (2p). Rulați exemplul din scheletul de laborator.

Task 1 (4p). Implementați, folosind cele 8 LED-uri de pe placa de laborator, **o secvență simplă** și **o secvență avansată** de aprindere dintre cele de mai jos. Selecția secvenței se face prin intermediul a două butoane conectate la uC pe pinii PB2 și PD6 (PB2 selectează secvența simplă, PD6 selectează secvența avansată).

```
      Simplu1          Simplu2          Simplu3
 t00  *-------         *-------         *------*
 t01  -*------         **------         -*----*-
 t02  --*-----         ***-----         --*--*--
 t03  ---*----         ****----         ---**---
 t04  ----*---         *****---         ---**---
 t05  -----*--         ******--         --*--*--
 t06  ------*-         *******-         -*----*-
 t07  -------*         ********         *------*
      mergi la t00     mergi la t00     mergi la t00


      Avansat1         Avansat2         Avansat3
 t00  *-------         *-------         *-------
 t01  -*------         **------         -*------
 t02  --*-----         ***-----         --*-----
 t03  ---*----         ****----         *--*----
 t04  ----*---         *****---         -*--*---
 t05  -----*--         ******--         --*--*--
 t06  ------*-         *******-         *--*--*-
 t07  -------*         ********         -*--*--*
 t08  ------*-         -*******         --*--*--
 t09  -----*--         --******         mergi la t06
 t10  ----*---         ---*****
 t11  ---*----         ----****
 t12  --*-----         -----***
 t13  -*------         ------**
 t14  *-------         -------*
      mergi la t00     mergi la t00
```

Task 2 (4p). Implementţati codul Morse folosind speaker-ul. Veţi implementa 2 tonuri, unul scurt cu durata de 50 ms şi unul lung cu durata de 150 ms, cu pauză de 50 ms între 2 tonuri consecutive. Selecţia între tonuri se va face folosind cele 2 butoane de pe PB2 (produce un ton scurt) şi PD6 (produce un ton lung). Speaker-ul este conectat la PD4.

Task 3 (Bonus 2p). Scrieți un program care produce 8 note diferite folosind speaker-ul. Inițial speaker-ul va fi oprit. La apăsarea butonului de pe PD6 programul va trece la nota superioară. Dacă era la cea mai înaltă notă (a 8-a), programul nu va face nimic. La apăsarea butonului de pe PB2 programul va trece la nota inferioară. Dacă era la cea mai joasă notă notă (prima), programul va opri speaker-ul. Nota curentă va fi afișată prin intermediul celor 8 LED-uri, aprinzând LED-ul corespunzător notei produse (LED-ul 1 va fi aprins pentru nota 1, LED-ul 2 pentru nota 2, etc. Când speaker-ul este oprit niciun LED nu va fi aprins). Mai jos aveți funcționarea programului exemplificată în pseudocod.

```
frecventa_nota[8] = {262, 294, 320, 349, 392, 440, 494, 523};

init:
    nota_curenta = -1
    butoane.init()
    speaker.init()
    led.init()

bucla_principala:
    if (butoane.apasat(PD6) && nota_curenta < 7)
        ++nota_curenta;

    if (butoane.apasat(PB2) && nota_curenta > -1)
        --nota_curenta;

    if (nota_curenta != -1)
        speaker.reda(nota_curenta)
        led.aprinde(nota_curenta)
    else
        speaker.opreste()
        led.opreste()

    goto bucla_principala
```

6. Resurse

- Schelet de cod
- Soluție laborator - disponibilă începând de săptămâna viitoare

- Datasheet ATmega324
- Responsabili: Marius Andrei [mailto:mariuslucianandrei@gmail.com]

7. Link-uri utile

- AVR freaks [http://www.avrfreaks.net/]
- AVR libc [http://www.nongnu.org/avr-libc/user-manual/modules.html]
- AVR GCC [https://gcc.gnu.org/wiki/avr-gcc]
- Detalii introductive despre programarea pe AVR (blog) [http://hackaday.com/2010/10/23/avr-programming-introduction/]
- Cum funcționează LED-urile [http://electronics.howstuffworks.com/led.htm]

pm/lab/lab0.txt · Last modified: 2020/02/27 22:08 by dorin_marian.ionita

Laboratorul 1: USART. Digital Debugging

Ca în toate lucrările de inginerie, bug-uri pot și vor apărea si în sisteme incorporate. În condiții de funcționare, este important să avem o modalitate de comunicare cu dispozitivul integrat. Pentru o imagine de ansamblu asupra metodelor de depanare posibile, vom face o scurtă introducere, apoi vom studia în detaliu interfața serială USART, folosită în mod uzual pentru comunicația serială dintre două dispozitive.

1. Ce este diferit fata de depanarea uzuala?

Motivul pentru care depanarea embedded este mai dificilă decât depanarea obișnuită a software-ului provine din mai multe probleme:

- Un începător este obișnuit cu instrumente de nivel înalt generice, de cele mai multe ori gratuite (de ex. IDE-uri, debuggere etc). Pentru sistemele embedded, este posibil ca user-ul sa nu aibă acces la instrumente hardware si software dedicate.
- Dacă nu aveți un debugger specializat și încercați să utilizați unul generic (să spunem Remote GDB), ce vă face să credeți că aveți stiva de rețea necesară pentru acel debugger? Dacă ați implementat o astfel de stivă, cât de sigur sunteți că funcționează corect?
- Chiar dacă ați avut un debugger specializat (de exemplu, o sondă Lauterbach), cel mai probabil veți avea nevoie de configurații specializate pentru debugger, pentru a lucra cu hardware-ul vostru (de exemplu, Practice Scripting Language for Lauterbach T32)
- Depanarea invazivă poate afecta comportamentul codului vostru - gândiți-vă la RTOS (sisteme de operare în timp real), SMP (sisteme multiprocesor) sau la circuitul dvs. (de exemplu, modificați circuitul pentru a măsura intensitatea curentului).
- Hardware-ul dvs. ar putea avea erori.
- Chiar și tipărirea mesajelor de eroare ar putea să nu funcționeze, deoarece uneori trebuie să implementați o astfel de funcție și ar putea avea bug-uri în sine.

Cu toate acestea, principiile de debugging sunt aceleasi ca si în cazul software-ul de nivel înalt: Trebuie sa comparați ce se dorește de la sistem (cod / circuit) cu ceea ce, de fapt, face si, pentru asta, ai nevoie de vizibilitate.

2. Instrumente necesare

Vizibilitatea la nivel hardware se realizează printr-o formă de InputOutput (dacă este disponibilă):

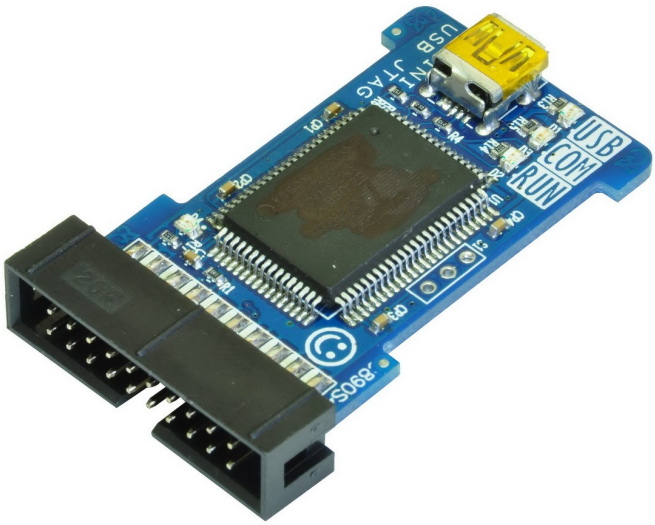
- Depanare LED - verificarea stării true/false
- **mesaje pe interfața serială / USART - depanare prin USART, bluetooth, etc**
- Debuggere avansate pentru afisarea si modificarea memoriei / registrelor (a se vedea JTAG de mai jos)
- Loopbacking (conectarea ieșirilor la intrări) poate oferi informații despre modul în care comenzile sunt trimise dispozitivelor externe.

Instrumente de măsurare:

- Multimetre (pentru valori statice)
  - Rezistență: Doar conectați o componentă de circuit între sonde.
  - Tensiune: conectați în paralel - sonda pozitivă (roșie) la punctul potențial mai mare, sonda negativă (neagră) la punctul potențial mai mic. Pentru potențialul într-un singur punct: sonda negativă la GND, sonda pozitivă în punctul dorit.
  - Curent: conectați-vă în serie.
  - Începeți întotdeauna prin a seta scara multimetrului la maxim, apoi corectați progresiv scara până când aveți cea mai precisă măsurare.
  - Pentru a verifica polaritatea diodelor: Selectați verificare diodă. Daca plasați sonda negativă pe anod și sonda pozitivă pe catod, multimetrul va face un sunet.
  - Pentru a verifica scurtcircuite/conectivitate: Utilizați verificare diodă – daca aparatul emite un sunet atunci înseamnă că există un scurtcircuit/conexiune între acele două puncte.



- Osciloscoape
- Analizoare logice (pentru semnale digitale)
- Analizoare de protocol (pentru protocoale încorporate, ar fi I2C, SPI, etc)
- Debuggere bazate pe JTAG - pot avea funcții foarte avansate: citire/modificare memorie, interoperarea cu kernelul, oprire ceas sistem, etc



Exemplu de flux de depanare

Un exemplu de flux de depanare ar putea fi următorul:

- Verificați de două ori foaia de catalog și schema. Accesați registrele potrivite? Sunt perifericele conectate la pinii la care am crezut că sunt?
- Este posibilă depanarea prin mesaje de debug (echivalent printf)?:
- Ai o stivă ethernet? Dacă da, ia în considerare SSH, NFS, etc pentru a realiza debugging printf.
- În cazul în care aveti o stiva de rețea, poate sunt disponibile protocoale mai simple: UART? Putem conecta ceva prin UART? Poate un PC, modul HC-05 Bluetooth, un LCD?
- Se poate lua în considerare depanarea cu LED-uri.

- Izolați problema folosind instrumente de măsura: multimetre, osciloscoape, analizoare logice.
- Încercați un debugger JTAG sau orice alt depanator despre care știți că funcționează cu dispozitivul dvs.

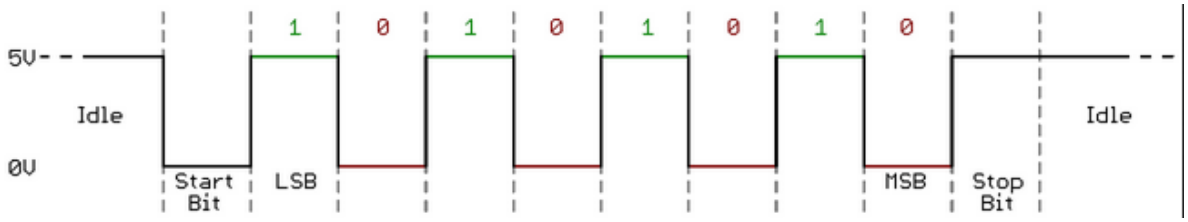
3. Interfața serială USART

Interfața serială este cel mai facil mod de a comunica cu microcontroller-ul vostru pentru citirea de date sau trimiterea de comenzi. Din perspectiva microcontroller-ului, comunicația serială se bazează pe doar **două linii de date**:

- linie pentru **transmisie**, notată **Tx**,
- linie pentru **recepție**, notată **Rx**.

Comunicația este full-duplex, se poate transmite concomitent cu recepția.

Transmisia asincronă de date se face la nivel de **cadre** (frames), fiecare cadru fiind format din mai mulți biți, având formatul descris în figură.



Se transmite un **bit de start**, apoi un cuvânt de date. Urmează un **bit de partitate**, opțional, cu rolul de a face o verificare simplă a corectitudinii datelor, și **unul sau doi biți de stop**.

Microcontroller-ul ATmega328p include un periferic **USART** (Universal Synchronous-Asynchronous Receiver/Transmitter) pentru interfața serială. În partea de inițializare a acestui periferic trebuie efectuați următorii pași:

- alegerea vitezei pentru transmisia de date - baud rate-ul (valori uzuale: 9600, 19200, 38400, 57600, 115200)
- alegerea formatului cadrului (câți biți de date, de stop, dacă va conține sau nu bit de partitate)
- activarea transmisiei și recepției datelor pe liniile RX și TX.

Baud rate este numărul de simboluri/pulsuri pe secundă al semnalului. În esență, reprezintă viteza de transmisie și este foarte important ca și transmițătorul și receptorul să folosească același baud rate pentru transmisia corectă a datelor. Una dintre cele mai comune probleme cu USART configurat în mod asincron este setarea diferită a baud rate-ului pe transmițător și pe receptor. Această neconcordanță se manifestă prin recepția unor date greșite (transmițătorul trimite caracterul 'a', receptorul primește caracterul '&')

Pentru ca două dispozitive, în cazul nostru PC-ul și placa de laborator, să poată comunica între ele prin USART în mod asincron, trebuie configurate **identic**. Dacă placa este configurată cu baud rate 115200, 9 biți de date, 1 bit de stop și fără paritate atunci PC-ul trebuie configurat **exact la fel** pentru a comunica.

3.1 Registre

Descrierea completă pentru:

- cele trei registre de control
- registrul pentru baud rate
- buffer-ele de transmisie/recepție

Se gaseste în Datasheet Atmega 328p în capitolul 19. Registrele au un 'n' la sfârșit care distinge între mai multe periferice USART ce pot exista pe un microcontroller (pe ATmega328P 'n' va lua doar valoarea 0 corespunzatoare USART0).

USART Data Register n (UDRn)

Bit	7	6	5	4	3	2	1	0	
	RXB[7:0]								UDRn (Read)
	TXB[7:0]								UDRn (Write)
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

RXB și TXB sunt buffer-ele de recepție, respectiv transmisie. Ele folosesc *aceeași adresă de I/O*. Deci RXB este accesat citind din UDRn, TXB scriind în UDRn. Buffer-ul de transmisie poate fi scris numai atunci când bitul UDRE (*USART Data Register Empty*) din portul UCSRnA este 1. În caz contrar, scrierile vor fi ignorate.

USART Control and Status Register n A (UCSRnA)

Bit	7	6	5	4	3	2	1	0	
	RXCn	TXCn	UDREn	FEn	DORn	UPEn	U2Xn	MPCMn	UCSRnA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

UCSRnA este registrul de stare al controller-ului de comunicație. Biții cei mai importanți sunt:

- **RXCn** – **Receive Complete** – devine 1 când există date primite și necitite. Când buffer-ul de recepție este gol, bitul este resetat automat
- **TXCn** – **Transmit Complete** – devine 1 când buffer-ul de transmisie devine gol
- **UDREn** – **Data Register Empty** – devine 1 când buffer-ul de transmisie poate accepta noi date

USART Control and Status Register n B (UCSRnB)

Bit	7	6	5	4	3	2	1	0	
	RXCIEn	TXCIEn	UDRIEn	RXENn	TXENn	UCSZn2	RXB8n	TXB8n	UCSRnB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

UCSRnB este un registru de control. Biții importanți:

- **RXCIEn** – **Receive Complete Interrupt Enable** – când este 1, controller-ul de comunicație va genera o întrerupere când au fost primite date
- **TXCIEn** – **Transmit Complete Interrupt Enable** – când este 1, controller-ul de comunicatie va genera o întrerupere când buffer-ul de transmisie devine gol
- **UDRIEn** – **Data Register Empty Interrupt Enable** – când este 1, controller-ul de comunicație va genera o întrerupere când buffer-ul de transmisie mai poate accepta date
- **RXENn** – **Receiver Enable** – dacă este 0, nu se pot recepta date
- **TXENn** – **Transmitter Enabler** – dacă este 0, nu se pot transmite date
- **UCSZn2** – împreună cu UCSZ1 și UCSZ0 din portul UCSRC, selectează dimensiunea unui cuvânt de date

USART Control and Status Register n C (UCSRnC)

Bit	7	6	5	4	3	2	1	0	
	UMSELn1	UMSELn0	UPMn1	UPMn0	USBSn	UCSZn1	UCSZn0	UCPOLn	UCSRnC
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	1	1	0	

UCSRnC este tot un registru de control. Biții importanți:

- **UMSELn** – **Mode Select** – 0 pentru funcționare asincronă, 1 pentru funcționare sincronă

- **UPMn1, UPMn0** – **Parity Mode** - Fiind vorba de doi biți, împreună pot avea 4 valori posibile, detaliate în tabelul ce urmează:

UPMn1	UPMn0	Parity Mode
0	0	Disabled
0	1	Reserved
1	0	Enabled, Even Parity
1	1	Enabled, Odd Parity

- **USBSn** – **Stop Bit Select** – 0 pentru un bit de stop, 1 pentru doi biți de stop

USBSn	Stop bit(s)
0	1-bit
1	2-bit

- **UCSZn1, UCSZn0** – împreună cu UCSZn2 din portul UCSRnB, selectează dimensiunea cuvântului de date

UCSZn2	UCSZn1	UCSZn0	Character Size
0	0	0	5-bit
0	0	1	6-bit
0	1	0	7-bit
0	1	1	8-bit
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Reserved
1	1	1	9-bit

USART Baud Rate Registers (UBRRn)

Bit	15	14	13	12	11	10	9	8	
	–	–	–	–	UBRR[11:8]				UBRRnH
	UBRR[7:0]								UBRRnL
	7	6	5	4	3	2	1	0	
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

UBRRn este registrul care selectează **baud rate**-ul. Are 12 biți. Primii 4 se află în UBRRnH, ceilalți 8 în UBRRnL. Valoarea pe care o scriem în UBRRn depinde de frecvența procesorului și de baud rate-ul dorit. În tabelul următor găsiți valorile pentru cele mai uzuale viteze de transmisie pentru o frecvență de ceas de 16 MHz.

Baud Rate (bps)	f_{osc} = 16.0000MHz			
	U2Xn = 0		U2Xn = 1	
	UBRRn	Error	UBRRn	Error
2400	416	–0.1%	832	0.0%
4800	207	0.2%	416	–0.1%
9600	103	0.2%	207	0.2%
14.4k	68	0.6%	138	–0.1%
19.2k	51	0.2%	103	0.2%
28.8k	34	–0.8%	68	0.6%
38.4k	25	0.2%	51	0.2%
57.6k	16	2.1%	34	–0.8%
76.8k	12	0.2%	25	0.2%
115.2k	8	–3.5%	16	2.1%
230.4k	3	8.5%	8	–3.5%
250k	3	0.0%	7	0.0%
0.5M	1	0.0%	3	0.0%
1M	0	0.0%	1	0.0%
Max.(1)	1Mbps		2Mbps	

Note: 1. UBRRn = 0, error = 0.0%

Este de dorit alegerea unui baud rate care să poată fi obținut exact din frecvența de ceas. În caz contrar se definește o toleranță (eroarea maximă a baud rate-ului) pentru care comunicația se poate realiza în condiții acceptabile. Dacă doriți să aprofundați subiectul, găsiți multe informații aici [https://www.allaboutcircuits.com/technical-articles/the-uart-baud-rate-clock-how-accurate-does-it-need-to-be/]

3.2 Exemplu de utilizare

```
void USART0_init()
{
  /* seteaza baud rate la 9600 */
  UBRR0 = 103;

  /* porneste transmitatorul */
  UCSR0B = (1 << TXEN0) | (1 << RXEN0);

  /* seteaza formatul frame-ului: 8 biti de date, 1 bit de stop, fara paritate */
  UCSR0C &= ~(1 << USBS0);
  UCSR0C |= (3 << UCSZ00);
}

void USART0_transmit(unsigned char data) {
    /* așteaptă până când buffer-ul e gol */
    while(!(UCSR0A & (1<<UDRE0)));

    /* pune datele în buffer; transmisia va porni automat în urma scrierii */
    UDR0 = data;
}

char USART0_receive()
{
  /* asteapta cat timp bufferul e gol */
  while (!(UCSR0A & (1 << RXC0)));

  /* returneaza datele din buffer */
  return UDR0;
}
```

Scrieri pe 16 biți

(3 << x)

Pentru biți de configurație care se găsesc întotdeauna unul după altul se folosește și o mască cu mai mulți biți shiftați cu index-ul celui mai din dreapta: `(3 << UCSZ00)` înlocuiește astfel `(1 << UCSZ01) | (1 << UCSZ00)`

(1 << x) | (1 << y) De cele mai multe ori o să facem măști compuse, pe care le vom aplica unui registru I/O în același timp. **Atenție!** Pot doar să compun măști pentru aceeași operație, nu pot aplica o mască *SAU* în același timp cu o mască *ȘI* pentru că rezultatul ar fi complet eronat!

3.3 Utilizarea interfetei seriale de pe Arduino UNO

Arduino UNO se conecteaza la PC prin intermediul interfetei seriale, dar utilizeaza un convertor USB-UART integrat pe placa. Prin intermediul acestei interfete si utilizand IDE-ul dedicat Arduino se poate programa microcontrollerul, dar se poate asigura si un canal de debug. Astfel, prin mesaje simple, se poate afla starea sistemului, se pot afisa valorile variabilelor, sau chiar se pot trimite comenzi, interfata functionand bidirectional. Mai multe detalii se pot gasi aici [https://www.arduino.cc/reference/en/language/functions/communication/serial/]

Configurația implicită pentru interfața serială USART folosește 8 biți de date, un bit de stop, fără paritate (8N1).

Următorul program poate trimite mesaje de la Arduino către PC, prin USB (sau folosind emulatorul din Tinkercad)

```
void setup()
{
  Serial.begin(9600);
  Serial.println("in function setup");
}

void loop()
{
  Serial.println("in function loop");
  delay(1000);
}
```

Următorul program poate primi mesaje trimise de PC, prin USB (sau folosind emulatorul din Tinkercad)

```
void setup()
{
  Serial.begin(9600);
  Serial.println("astept comenzi");
}

void loop()
{
  if (Serial.available()) {
    char a = Serial.read();
    char buf[20];
    sprintf(buf, "%s: %c", "primit caracter", a);
    Serial.println(buf);
  }
}
```

4. Exerciții

Task 1 (3p)

Folosind interfața serială și biblioteca Arduino, trimiteți următoarele comenzi dintr-un terminal serial:

- "on" - aprinde un led
- "off" - stinge un led
- "blink" – blink un led la interval de 200 ms
- "get" - afișează prin interfața serială starea unui buton

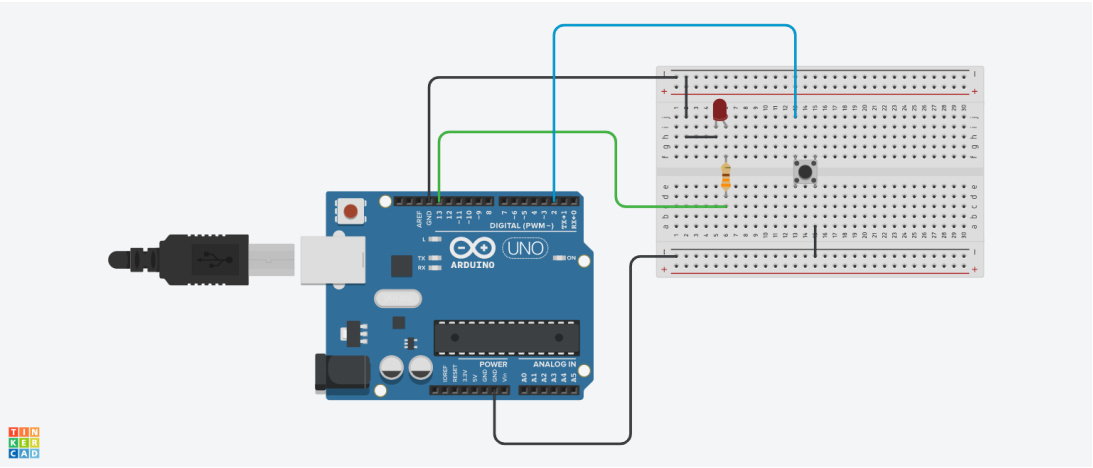
Fiecare comandă va fi urmată de caracterul de control: "\n" (newline). Configurați terminalul serial să trimită (doar) acest caracter după fiecare mesaj. Implementați un program în Arduino care să recunoască aceste comenzi și să realizeze acțiunile corespunzătoare pentru fiecare dintre ele. Folosiți pin-ul 13 (PB5) pentru LED și pin-ul 2 (PD2) pentru buton.

- Comanda blink va porni funcția blink (blink led "la infinit")
- Comenzile on/off vor opri funcția blink.

- pentru a primi un șir de caractere de la interfața serială, puteți folosi un vector de caractere pe post de buffer
- pentru a verifica dacă șirul de caractere primit corespunde cu comanda, puteți folosi funcția *strcmp()*, ex. *strcmp(a, b) == 0*

Pentru a putea primi comenzi pe serială, este de dorit să nu existe delay-uri sau funcții blocante în programul principal (în loop). Funcția blink ar trebui să fie non-blocantă. Vom folosi funcția *millis()* pentru a măsura durata de timp (în loc să ținem programul blocat până expiră delay-ul) astfel:

```
long ts = 0; // global variable is your friend
void loop() {
 // my other code here
 if ((millis() - ts) >= 100) {
    ts = millis();
    // my non-blocking timed loop here
 }
 // my other code here
}
```



Task 2 (3p)

Rulați exemplul pentru USART pe bază de registre. Pentru configurările serialei, urmăriți fisierul usart.c din schelet.

Schelet

Capitole utile din Datasheet Atmega 328p

- 1. Pin Configurations - pag. 3
- 19. USART - pag. 143
  - secțiunile 19.1 - 19.2 pentru overview
  - secțiunea 19.3 pentru generarea ceasului
  - secțiunea 19.4 pentru formatul frame-ului
  - secțiunile 19.5 - 19.8 pentru exemple de cod - inițializare și funcționare
  - secțiunea 19.10 este referința pentru registrele I/O

Configurați USART0 cu următorii parametri: baud rate 19200, 8 biți de date, 1 bit de stop, fara paritate. Transmiteți către PC câte un mesaj pentru fiecare eveniment de apăsare/lăsare a unui buton (ex: se apasă PD2, se transmite "PD2 apăsat", se lasă PD2, se transmite "PD2 lăsat", câte o singură dată pe apăsare).

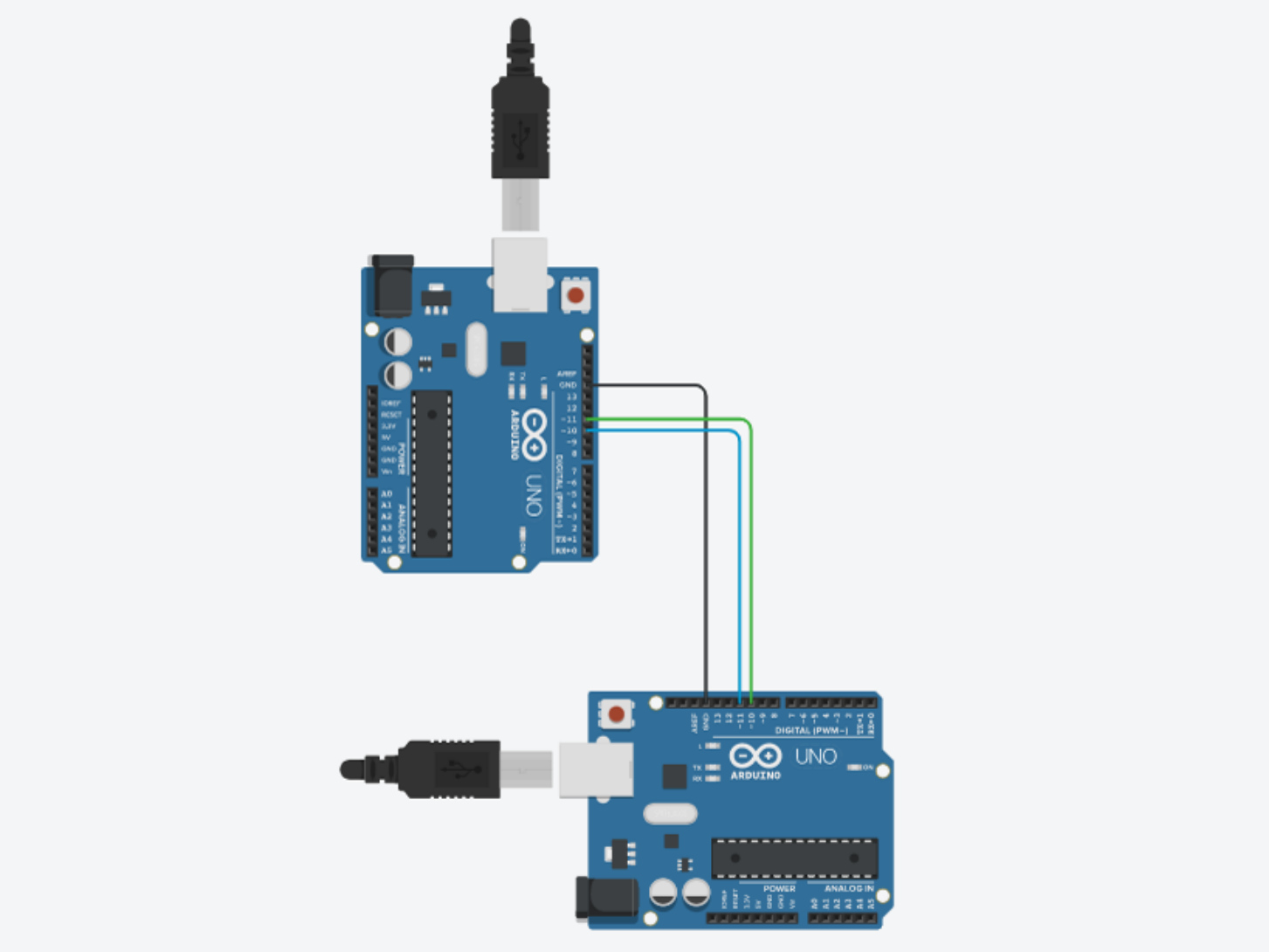
Terminalul serial din Arduino este configurat implicit în mod 8N1, doar baud rate-ul fiind configurabil. Pentru a avea control mai precis asupra formatului se poate folosi un terminal serial dedicat, precum Putty [https://www.putty.org/].

Task 3 (4p)

Colaborați cu colegii de lângă voi pentru a conecta două plăci Arduino între ele și trimiteți mesaje, dinspre primul Arduino către al doilea, și invers.

- Mesajele trimise de primul Arduino vor fi afisate în terminalul Serial conectat la cel de-al doilea Arduino, și invers.
- Aprindeți led-ul (PB5) pentru 500 ms atunci când este recepționat un mesaj. Pentru a nu bloca programul (și implicit recepționarea mesajelor), nu folosiți funcția delay.
- Atenție la realizarea conexiunilor RX/TX (RX1 la TX2, TX1 la RX2) și GND (GND1 la GND2, dacă plăcile Arduino sunt conectate la PC-uri diferite)

- În caz că firele nu sunt suficient de lungi pentru a conecta direct cele două Arduino (legate în același timp la 2 PC-uri prin cabluri USB), folosiți breadboard-urile și realizați conexiunile prin mai multe fire.



Până acum am folosit interfața serială pentru a realiza comunicația dintre Arduino și PC, prin intermediul adaptorului USB-UART disponibil pe placă. Există însă situații în care se dorește conectarea unui dispozitiv extern la Arduino (de ex. un modul de comunicație radio, bluetooth, GPS, sau la modul general un alt microcontroller).

În cazul în care nu avem nevoie de conexiunea la PC, putem conecta un al doilea dispozitiv pe liniile RX și TX. Dacă însă am vrea să facem debug sau să trimitem mesaje de la PC, conexiunea nu mai funcționează corect, deoarece interfața serială nu permite conectarea a mai mult de 2 dispozitive (cel puțin pe linia TX).

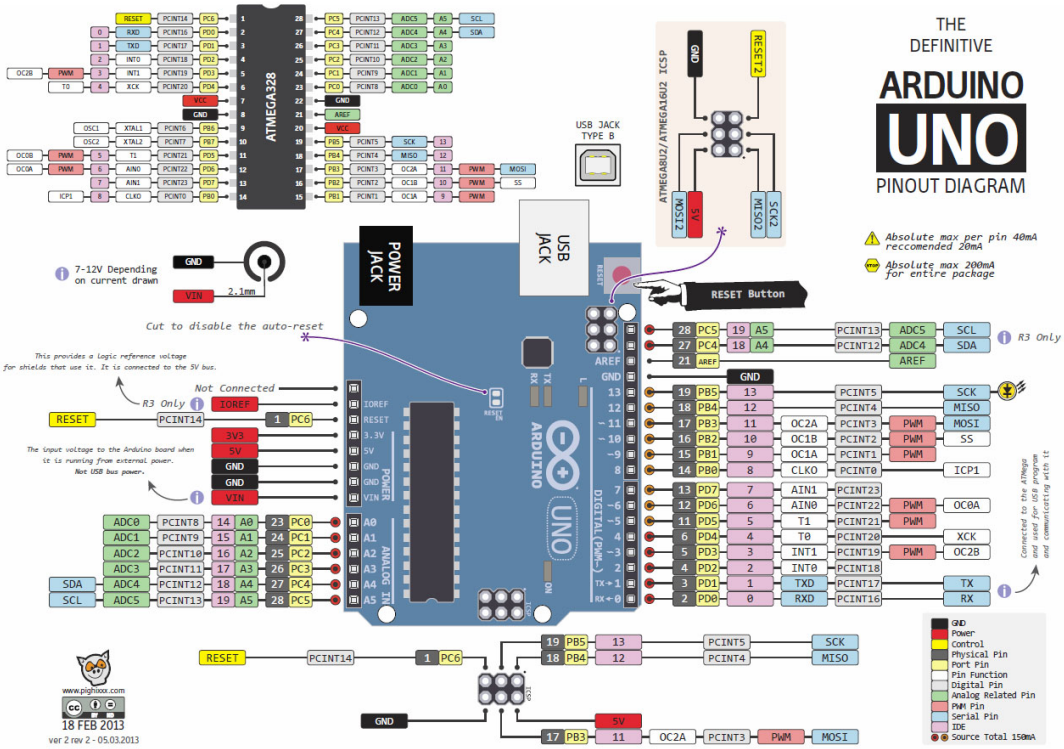
O altă constrângere este că avem o singură interfață USART pe ATmega328p, fapt pentru care vom avea nevoie de simularea unei a doua interfețe seriale în software.

Găsiți un exemplu în Arduino IDE: Files > Examples > SoftwareSerial > SoftwareSerialExample

Bonus (2p) Realizați implementarea Task 1 pe bază de registre.

5. Resurse

- Datasheet Atmega 328p
- Arduino UNO pinout



- Responsabil: Alexandru Predescu [mailto:alexandru.predescu@upb.ro]

Rezolvare

Laboratorul 2: Întreruperi hardware. Întreruperi externe

Acest laborator are ca scop familiarizarea voastră cu lucrul cu întreruperile hardware și în particular cu întreruperile externe. Vom folosi întreruperi externe pentru a detecta imediat (în timp real) apăsarea unui buton, independent de programul principal.

1. Întreruperi

O întrerupere hardware reprezintă un semnal sincron sau asincron de la un periferic ce semnalizează apariția unui eveniment care trebuie tratat de către procesor. Tratarea întreruperii are ca efect suspendarea firului normal de execuție al unui program și lansarea în execuție a unei **rutine de tratare a întreruperii (RTI)**.

Întreruperile hardware au fost introduse pentru a se elimina buclele pe care un procesor ar trebui să le facă în așteptarea unui eveniment de la un periferic. Folosind un sistem de întreruperi, perifericele pot comunica cu procesorul, acesta din urma fiind liber să-și ruleze programul normal în restul timpului și să își întrerupă execuția doar atunci când este necesar.

Înainte de a lansa în execuție o RTI, procesorul trebuie sa aibă la dispoziție un mecanism prin care să salveze starea în care se afla în momentul apariției întreruperii. Aceasta se face prin salvarea într-o memorie, de cele mai multe ori organizată sub forma unei stive, a registrului contor de program (Program Counter), a registrelor de stare precum și a tuturor variabilelor din program care sunt afectate de execuția RTI. La sfârșitul execuției RTI starea anterioară a registrelor este refăcută și programul principal este reluat din punctul de unde a fost întrerupt.

Pentru a asocia o întrerupere cu o anumită rutină din program, procesorul folosește **tabela vectorilor de întrerupere** (TVI), ilustrată în figura de mai jos. În această tabelă, fiecărei întreruperi îi este asociată adresa rutinei sale de tratare, la care programul va face salt în cazul apariției acesteia. Aceste adrese sunt predefinite și sunt mapate în memoria de program într-un spatiu contiguu care alcătuiește TVI. Adresele întreruperilor în TVI sunt setate în funcție de prioritatea lor: cu cât adresa este mai mică cu atât prioritatea este mai mare.

Vector no.	Program address	Source	Interrupt definition
1	0000	RESET	External Pin, Power-on Reset, Brown-out Reset, Watchdog Reset and JTAG AWR Reset
2	0002	INT0	External Interrupt Request 0
3	0004	INT1	External Interrupt Request 1
4	0006	PCINT0	Pin Change Interrupt Request 0
5	0008	PCINT1	Pin Change Interrupt Request 1
6	000A	PCINT2	Pin Change Interrupt Request 2
7	000C	WDT	Watchdog Time-out Interrupt
8	000E	TIMER2_COMPA	Timer/Counter2 Compare Match A
9	0010	TIMER2_COMPB	Timer/Counter2 Compare Match B
10	0012	TIMER2_OVF	Timer/Counter 2 Overflow
11	0014	TIMER1_CAPT	Timer/Counter1 Capture Event
12	0016	TIMER1_COMPA	Timer/Counter1 Compare Match A
13	0018	TIMER1_COMPB	Timer/Counter1 Compare Match B
14	001A	TIMER1_OVF	Timer/Counter1 Overflow
15	001C	TIMER0_COMPA	Timer/Counter0 Compare Match A
16	001E	TIMER0_COMPB	Timer/Counter0 Compare Match B
17	0020	TIMER0_OVF	Timer/Counter0 Overflow
18	0022	SPI_STC	SPI Serial Transfer Complete

19	0024	USART0_RX	USART0 Rx Complete
20	0026	USART0_UDRE	USART0 Data Register Empty
21	0028	USART0_TX	USART0 Tx Complete
22	002A	ADC	ADC Conversion Complete
23	002C	EE_READY	EEPROM Ready
24	002E	ANALOG_COMP	Analog Comparator
25	0030	TWI	Two-Wire Serial Interface
26	0032	SPM_READY	Store Program Memory Ready

La apariția unei întreruperi, în afară de salvarea stării, procesorul dezactivează întreruperile, iar la revenirea din rutina de tratare le reactivează. Activarea lor poate fi realizată forțat și din handler-ul de întrerupere (de exemplu, suntem în handler-ul pt recepția unui frame pe interfața seriala, și dorim să activăm întreruperile unui timer).

Utilizarea întreruperilor

În general, pașii parcurși pentru a activa o întrerupere sunt următorii:

Pornim mecanismul de tratare a întreruperilor

Mecanismul de întreruperi trebuie activat explicit (bitul I - Global Interrupt Enable din registrul SREG). Pentru a seta și reseta acest bit, există și două funcții ajutătoare:

```
// activează întreruperile
sei();
// dezactivează întreruperile
cli();
```

În general, pașii parcurși pentru a activa o întrerupere externă INT sau PCINT sunt următorii:

Configurăm perifericul care va trimite întreruperile

În exemplu, pentru INT0 și INT1 (pinii PD2 și PD3), configurarea se face din registrul EICRA (External Interrupt Control Register A, secțiunea 12.2.1, pagina 54 din datasheet, în timp ce pentru întreruperi de tip pin change se folosește registrul PCICR.

```
// întreruperi externe
EICRA |= (1 << ISC00);    // set INT0 to trigger on ANY logic change
// întreruperi de tip pin change (activare vector de întreruperi)
PCICR |= (1 << PCIE2); // enable the pin change interrupt, set PCIE2 to enable PCMSK2 scan
// alte întreruperi
```

O întrerupere externă poate fi configurată astfel încât **să detecteze tranziția semnalului logic** pe front crescător, descrescător sau oricare (vezi tabelul 13-1 din datasheet). În comparație, o întrerupere de tip pin change este generată la modificarea **nivelului logic** al semnalului, iar valoarea semnalului **trebuie verificată explicit în codul întreruperii**.

Scriem rutina de tratare a întreruperii

De exemplu, pentru întreruperile de tip INT0 și respectiv PCINT2, codul va arăta în felul următor:

```
ISR(INT0_vect)
{
  // cod întrerupere externă PD2 /INT0
  // verificare tranziție pe front crescător, descrescător sau oricare
  // (după cum este configurat INT0)
}
```

```
ISR(PCINT2_vect){
  // cod întrerupere de tip pin change
  if ((PIND & (1 << PD4)) == 0){
     // întreruperea a fost generată de pinul PD4 / PCINT20
     // verificăm nivelul logic al pinului
  }
  // întreruperea a fost generată de alt pin
}
```

Dacă avem mai multe întreruperi pe același vector și de ex. mai multe butoane, nu se poate determina cu exactitate care dintre butoane a generat întreruperea de tip PCINT. În plus, este posibil să apară mai multe întreruperi în timp ce starea unui pin verificat să rămână neschimbată, caz în care va fi detectată o nouă apăsare în mod eronat.

Activăm întreruperea și testăm programul

În cazul de față, pentru activarea întreruperilor externe, se configurează registrul External Interrupt Mask Register (`EIMSK`) astfel:

- biții `INT1:0` controlează dacă întreruperile externe (INT0-INT1) sunt activate

Pentru celelalte întreruperi, se configurează registrul corespunzător (e.g. `PCMSK2` pentru activarea întreruperilor de tip pin change pe fiecare pin de pe portul D)

```
// întrerupere externă
EIMSK |= (1 << INT0);     // Turns on INT0
// întrerupere de tip pin change
PCMSK2 |= (1 << PCINT20); // Turns on PCINT20 (PD4)
// activare întreruperi globale
sei();
```

Lucrul cu întreruperi

Reguli de programare în context întrerupere:

- Nu există o valoare de return. La închierea execuției handler-ului, procesorul execută din nou instrucțiuni de unde rămăsese înainte de declanșarea întreruperii
- Datorită faptului că handler-ul întârzie orice altă activitate din main și inhibă execuția altor întreruperi, se dorește ca **timpul de execuție să fie cât mai mic**.
- La folosirea unor variabile comune în mai multe handler-e de întrerupere și/sau main trebuie avut în vedere **accesul concurent** la acestea (în special la variabilele de 16/32 biți a căror modificare necesită mai mulți cicli de procesor).
- Variabilele comune trebuie marcate ca `volatile` pentru ca accesele la acestea să nu fie optimizate de către compilator

Alte wrapper-e (în jurul unor instrucțiuni ale core-ului AVR) oferite de interfață sunt:

- **sei()** - setează pe 1 bitul I din SREG, activând întreruperile globale. Apelează instrucțiunea assembler *sei*
- **cli()** - setează pe 0 bitul I din SREG, dezactivând întreruperile globale. Apelează instrucțiunea assembler *cli*
- **reti()** - întoarcerea dintr-o rutină de tratare a intreruperii, activează întreruperile globale. Ar trebui sa fie ultima instrucțiune apelată într-o rutină care folosește flag-ul ISR_NAKED.

Întreruperile întâlnite care nu au o rutină de tratare vor cauza o întrerupere de reset!

2. Întreruperi externe. INT vs. PCINT

Întreruperile se pot împărți în două categorii:

- întreruperile componentelor *interne*: timere, interfețe seriale (USART), convertor analog-digital
- întreruperile componentelor *externe*: activate de pinii externi ai uC-ului

Pe lângă întreruperile componentelor interne uC-ului, există și câteva linii pentru întreruperi de la periferice externe: INT0-INT1 și PCINT0-PCINT2. Diferența dintre aceste două tipuri de întreruperi externe este dată de capabilitățile suportate și de granularitatea lor.

Semnalele pentru întreruperile **INTn** pot genera o întrerupere la tranziție (crescătoare, descrescătoare sau ambele) sau pe nivel 0, în funcție de configurarea întreruperii. Întreruperile **PCINTn** (Pin Change INTerrupt) se declanșează la ambele tranziții (mai exact, atunci când se face toggle la valoare) și câte 8 pini sunt multiplexați pe o singură întrerupere. Semnalele de întrerupere PCINT se pot activa individual, însă pentru a afla exact ce semnal a declanșat o anumită întrerupere trebuie verificat registrul PINn corespunzător. Dacă mai multe semnale se declanșează în același timp, ele nu vor putea fi deosebite.

În cazul întreruperilor de tip pin change interrupt, a nu se confunda vectorul de tratare a întreruperilor ex. PCINT2 cu întreruperea efectivă ex. PCINT20 (PD4). Maparea dintre vector și întreruperi se poate găsi în secțiunea 30 pag. 278 din datasheet

3. Întreruperi externe în Arduino

Deși noi vom lucra în principal cu registre, în Arduino, întreruperile se mai pot configura folosind funcția **attachInterrupt()**. Funcția primește ca parametru numărul întreruperii externe (pe Arduino UNO avem INT0, INT1), funcția care va fi apelată (echivalent ISR), și modul prin care se face detecția tranziției:

- LOW: nivel logic 0
- CHANGE: tranziție de nivel logic
- RISING: front crescător
- FALLING: front descrescător

Dezactivarea întreruperilor externe se poate face cu funcția **detachInterrupt()** Activarea sau dezactivarea temporară a tuturor întreruperilor se poate face cu funcțiile **noInterrupts()** și **interrupts()**

```
int counter = 0;

void myCallback() {
  counter += 1;
}

void setup() {
  pinMode(2, INPUT_PULLUP);
  attachInterrupt(0, myCallback, FALLING);
}

void loop() {
  Serial.print("numar apasari: ");
  Serial.println(counter);
  delay(1000);
}
```

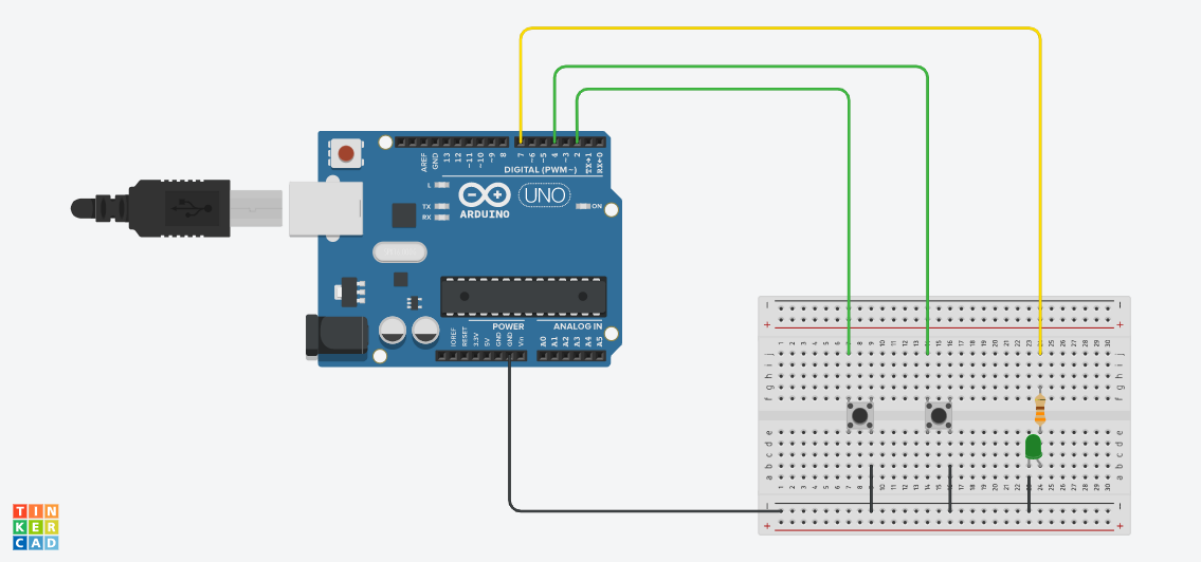
4. Exerciții

Task 1 (întreruperi / butoane) - 6p

Folosiți întreruperi externe (INT și/sau PCINT) pentru a detecta apăsarea unui buton conectat la PD2 (pin 2) și a unuia conectat la PD4 (pin 4). Modificați starea unui LED conectat la PD7 (pin 7) în ISR.

- Configurați întreruperea externă (INT) pe front descrescător (falling edge) sau PCINT-ul corespunzător pinului folosit
- Modificați starea LED-ului la apăsarea oricărui buton (PD2, PD4)

Tinkercad - Testați programul folosind următorul montaj:



Arduino - Încărcați codul pe placă și observați comportamentul. La apăsarea butonului, LED-ul își schimbă starea o singură dată? Adăugați o metodă de debouncing [https://www.arduino.cc/en/Tutorial/BuiltInExamples/Debounce] pentru a detecta o singură apăsare pe buton:

- NU este recomandat să folosiți delay în ISR
- Se poate folosi funcția *millis()* pentru a verifica intervalul de timp de la ultima apăsare (100 ms ar trebui să fie suficient pentru un pushbutton/microswitch, dar ajustați după caz)

```
ISR(INT0_vect)
{
  // cod întrerupere externă
  PORTD ^= (1 << PD7);
}

ISR(PCINT2_vect) {
  // cod întrerupere de tip pin change
  // TODO
}

void setup_interrupts() {
  // buton 1: PD2 / INT0
  // buton 2: PD4 / PCINT20
  cli();

  // input
  DDRD &= ~(1 << PD2) & ~(1 << PD4);
  // input pullup
  PORTD |= (1 << PD2) | (1 << PD4);

  // configurare intreruperi
  // intreruperi externe
  // TODO

  // întreruperi de tip pin change (activare vector de întreruperi)
  // TODO

  // activare intreruperi
  // intrerupere externa
  // TODO

  // întrerupere de tip pin change
```

```
  // TODO

  sei();
}

void setup() {
  setup_interrupts();
  DDRD |= (1 << PD7);
  PORTD &= ~(1 << PD7);
}

void loop() {

}
```

Task 2 (blink control) - 4p

Pe baza aceluiași montaj, folosiți întreruperi externe (INT și/sau PCINT) pentru a detecta apăsarea unui buton conectat la PD2 (pin 2) și a unuia conectat la PD4 (pin 4). Modificați intervalul de semnalizare (blink) al unui LED conectat la PD7 (pin 7) în ISR astfel:

- butonul conectat la PD2 crește intervalul cu 100 ms
- butonul conectat la PD4 scade intervalul cu 100 ms

```
volatile int dt = 500;

ISR(INT0_vect)
{
  // cod întrerupere externă
  // TODO
}

ISR(PCINT2_vect) {
  // cod întrerupere de tip pin change
  // TODO
}

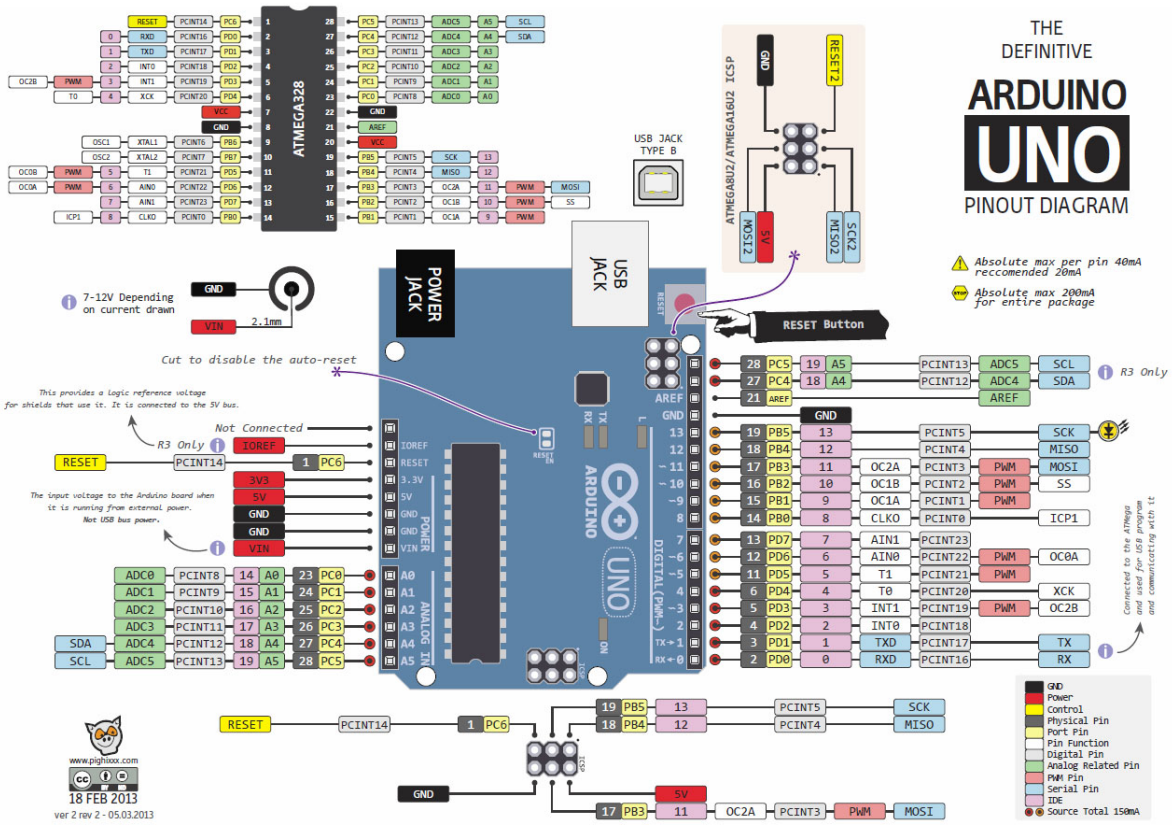
// setup_interrupts() la fel ca la Task 1

void setup() {
  setup_interrupts();
  DDRD |= (1 << PD7);
  PORTD &= ~(1 << PD7);
}

void loop() {
  PORTD |= (1 << PD7);
  delay(dt);
  PORTD &= ~(1 << PD7);
  delay(dt);
}
```

5. Resurse

- Datasheet Atmega 328p
- Arduino UNO pinout



- Responsabili: Alexandru Predescu [mailto:alexandru.predescu@upb.ro]

5. Linkuri utile

- AVR Libc - interrupt.h [http://www.nongnu.org/avr-libc/user-manual/group__avr__interrupts.html]
- Setul de instrucțiuni AVR [http://ww1.microchip.com/downloads/en/devicedoc/atmel-0856-avr-instruction-set-manual.pdf]
- TimerOne [https://www.arduino.cc/reference/en/libraries/timerone/]
- tone() [https://www.arduino.cc/reference/en/language/functions/advanced-io/tone/]
- Interrupts [https://www.renesas.com/us/en/support/engineer-school/mcu-programming-peripherals-04-interrupts]

Laboratorul 3: Timere. PWM

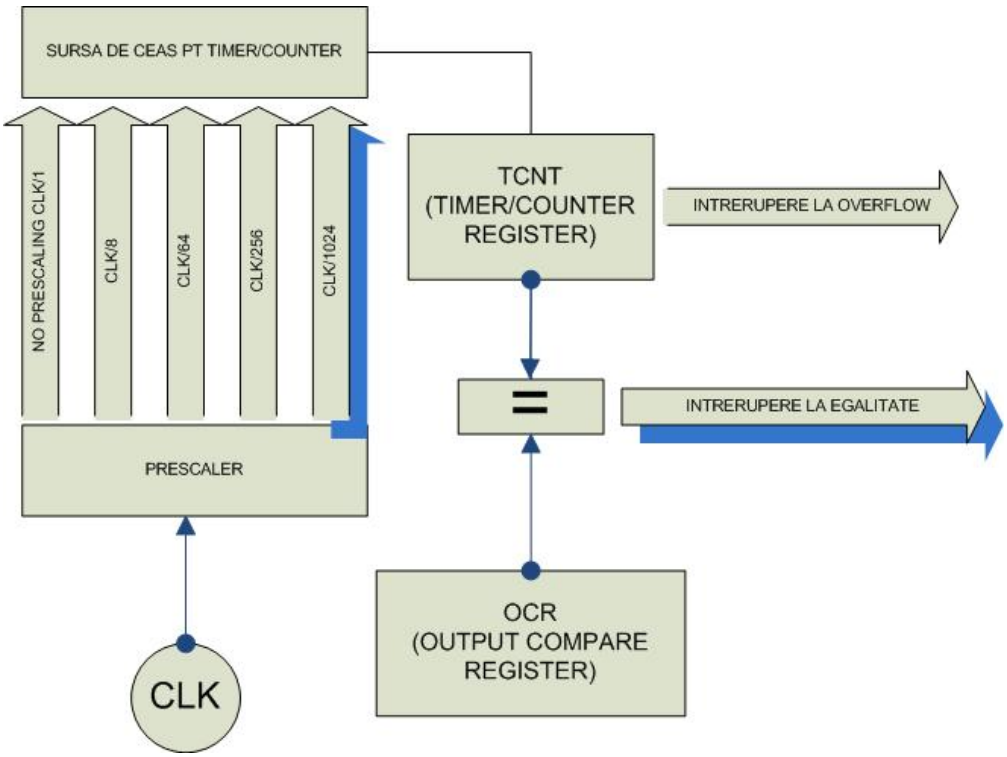
Acest laborator are ca scop familiarizarea voastră cu lucrul cu întreruperile hardware generate de timer-ele prezente în microcontroller-ul Atmega328p. Vom folosi timer-ele pentru a număra intervale de timp și pentru a genera semnale periodice. Folosind semnale PWM vom controla intensitatea luminoasă a unui LED și poziția unui servomotor.

1. Timere

1.1. Principiul de funcționare al unui Timer

Timer-ul/Counter-ul oferă facilitatea de a măsura intervale fixe de timp și de a genera întreruperi la expirarea intervalului măsurat. Un timer, odată inițializat va funcționa independent de unitatea centrală (UCP). Acest lucru permite eliminarea buclelor de delay din programul principal.

Componentele și funcționarea unui timer pentru ATmega328p pot fi descrise în linii mari de cele trei unități din figura de mai jos:



1. **Registrul numărător** (Timer Counter, TCNT) - măsoară efectiv intervalele de timp și este incrementat automat cu o frecvență dată.
2. **Prescaler-ul** - are rolul de a diviza în funcție de necesitățile aplicației frecvența de ceas.
3. La fiecare incrementare a TCNT are loc o **comparație între acest registru și o valoare stocată în registrul OCRn**. Această valoare poate fi încarcată de către programator prin scrierea registrului OCRn. Dacă are loc egalitatea se generează o întrerupere, în caz contrar incrementarea continuă.

Timer-ele sunt prevăzute cu mai multe canale astfel încât se pot desfășura diferite numărători în paralel. ATmega328p este prevăzut cu 3 unități de timer: două de pe 8 biți și una de 16 biți.

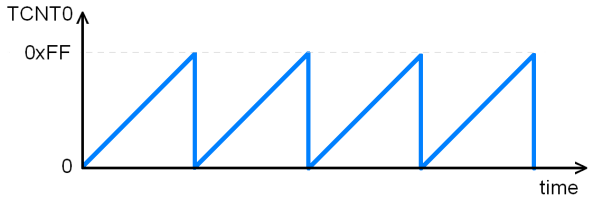
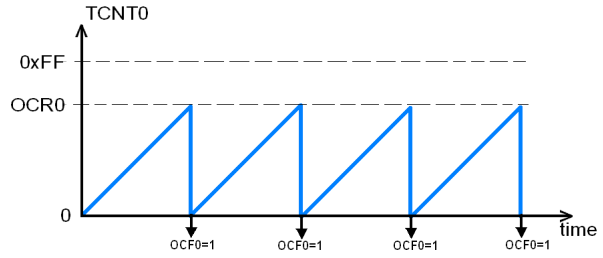
Timer-ele pot funcționa și în moduri PWM, astfel încat să genereze pe un pin de ieșire un semnal de comandă cu tensiune (medie) variabilă.

1.2. Moduri de funcționare

Timer-ele oferă mai multe moduri de funcționare (cu mici diferențe între Timer/Counter0, Timer/Counter1 și Timer/Counter2), ce se diferențiaza prin:

- valorile până la care se face incrementarea contorului
- modul în care se numără (doar crescător, sau alternativ crescător/descrescător)
- momentul când se face resetarea contorului

În următorul tabel sunt prezentate două moduri pe care le vom folosi în acest laborator.

Mod	Descriere	Imagine contor	Caracteristici
Normal	pornește de la 0 numără până la 0xFFFF		frecvența este fixată la câteva valori predefinite, date de frecvența ceasului și de prescaler
CTC Clear Timer on Compare	pornește de la 0 numără până când se atinge un prag (OCRnA pentru Timer 0/2, OCR1A sau ICR1 pentru Timer 1)		frecvența este variabilă, determinată de valoarea pragului, frecvența ceasului și de prescaler

Definiții care apar în datasheet:

- **BOTTOM**: capătul inferior din intervalul de numărare
- **TOP**: capătul superior al intervalului de numărare
- **MAX**: limita superioară a numărării, 255 (0xff) pentru 8 biți, 65535 (0xffff) pentru 16 biți. **TOP** poate fi **MAX** în anumite moduri de funcționare (ex: *Normal*).

1.3. Registre

Timer	Registre	Rol
Timer0 8 biți	`TCNT0`	Registrul contor al timer-ului 0 (cel care numără)
	`TCCR0A`, `TCCR0B`	Registre de control ale timer-ului 0 (diverși biți pentru configurarea timer-ului)
	`OCR0A`, `OCR0B`	Registre prag pentru timer-ul 0 (prag al numărătorii la care se poate declansa intreruperea, în funcție de configurare)
	`TIMSK0`, `TIFR0`	Registre cu biți de activare întreruperi timer 0 / flag-uri de activare (activați întreruperile)
Timer1 16 biți	`TCNT1`	Registrul contor al timer-ului 1 (la fel ca la timer0, doar că pe 16 biți)

	`TCCR1A`
	`TCCR1B`
	`TCCR1C`
		Registre control ale timer-ului 1 (la fel ca la timer0)

	`OCR1A` , `OCR1B`	Registre prag pe 16 biți ale timer-ului 1 (la fel ca la timer0)
	`TIMSK1`, `TIFR1`	(la fel ca la timer0)
	`ICR1`	Registru folosit pentru a reține valoarea contorului la apariția unui eveniment extern pe pin-ul `ICP` sau ca prag pentru unul din modurile CTC
Timer2 8 biți	aceleași registre ca la Timer0	Diferența față de Timer-ul 0 este posibilitatea folosirii unui oscilator extern separat pentru Timer-ul 2,

		pe pinii TOSC1 și TOSC2
	ASSR, GTCCR	Registre ce țin de funcționarea asicronă a acestui timer față de restul sistemului

Întreruperile sunt activate doar dacă bitul I din SREG este setat (întreruperile sunt activate global)

1.4. Lucrul cu Timer-ul

Setarea modului de funcționare

Pentru a configura un mod de funcționare, vom seta:

- biții WGM din timer-ul respectiv (care se găsesc în registrele TCCRnA din datasheet, la secțiunile aferente timerelor, "Register Description")
- pragul de numărare

De exemplu, ca să setăm timer-ul 0 să numere până la 5, ne vom uita în datasheet la capitolul 14 (*8-bit Timer/Counter0 with PWM*) - secțiunea *Register Description → TCCR0A*

1. găsim modul CTC ca având biții 0 1 0 pe biții WGM2..0
2. aflăm că modul CTC numără până la OCRA

Presupunând că plecăm de la un registru complet neinițializat (0 este valoarea default pentru majoritatea biților), avem următorul cod:

```
TCCR0A |= (1 << WGM01);
OCR0A = 5;
```

Setarea prescaler-ului

Pentru setarea prescaler-ului se vor modifica biții tip CS.. din registrul TCCRnB al timer-ului respectiv.

De exemplu, pentru setarea valorii de prescaler 256 pentru timer-ul 2, vom urmări în datasheet capitolul 17 (*8-bit Timer/Counter2 with PWM*) - secțiunea *Register Description → TCCR2B*

1. găsim tabelul pentru valorile CS..
2. aflăm că prescaler-ul 256 corespunde biților 1 1 0 pentru biții CS22 CS21 CS20

Presupunând că plecăm de la un registru complet neinițializat (0 este valoarea default pentru majoritatea biților), avem următorul cod:

```
TCCR2B |= (1 << CS22) | (1 << CS21);
```

Configurarea întreruperilor

```
// exemplu de configurare pentru Timer 1 în mod CTC, care va genera întreruperi cu frecvența de 2Hz
OCR1A = 31249;            // compare match register 16 MHz/256/2 Hz - 1
TCCR1B |= (1 << WGM12);   // CTC mode
TCCR1B |= (1 << CS12);    // 256 prescaler
```

Scriem rutina de tratare a întreruperii

Rutinele de tratare a întreruperii trebuie menționate în memorie la anumite adrese fixe (în vectorul de întreruperi). Pentru a face acest lucru folosim macro-ul de ISR(), care primește ca parametru tipul de întrerupere ce este tratată.

De exemplu, pentru întreruperile generate de Timer 1 în mod CTC, codul va arăta în felul următor:

```
// implementare rutină de tratare a întreruperii TIMER1_COMPA
ISR(TIMER1_COMPA_vect) {
  // cod întrerupere de la Timer1
}
```

Activăm întreruperea și testăm programul

```
// activare întrerupere TIMER1_COMPA
TIMSK1 |= (1 << OCIE1A);
// activare întreruperi la nivel global
sei();
```

Pentru un timer deja configurat, pentru a activa întreruperile trebuie doar să activăm bitul corespunzător din `TIMSKx`

De exemplu, pentru pragul A al timer-ului 1 vom scrie:

```
ISR(TIMER1_COMPA_vect) {
  // cod întrerupere
}

void configure_timer1() {
  // exemplu de configurare pentru Timer 1 în mod CTC
  // care va genera întreruperi cu frecvența de 2Hz
  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1 = 0;
  OCR1A = 31249;            // compare match register 16MHz/256/2Hz-1
  TCCR1B |= (1 << WGM12);   // CTC mode
  TCCR1B |= (1 << CS12);    // 256 prescaler
}

void init_timer1() {
  TIMSK1 |= (1 << OCIE1A);  // enable timer compare interrupt
}

void setup() {
  // dezactivăm întreruperile globale
  cli();
  configure_timer1();
  init_timer1();
  // activăm întreruperile globale
  sei();
}

void loop() {

}
```

1.5. Calculator

În general, pentru configurarea unui Timer, este necesară alegerea unui prescaler și a unei limite de numărare, în funcție de perioada dorită și de frecvența de lucru a procesorului (e.g. 16 MHz pentru Arduino UNO) și de modul de funcționare ales. Un exemplu de calcul este prezentat mai jos:

- calcul frecvență întreruperi în funcție de frecvența ceasului și a pragului de numărare:

```
f_int = f_clk / (prescaler * (tc + 1))
```

- calcul prag de numărare al timer-ului pentru a obține frecvența dorită:

```
tc = f_clk / (prescaler * f_int) - 1
```

Observăm că trebuie aleasă o valoare convenabilă pentru prescaler (din cele disponibile, ex. 8, 64, 256, 1024) și un prag de numărare (0-255 pentru timer-e pe 8 biți, 0-65535 pentru timer-e pe 16 biți) astfel

încât să se obțină frecvența exactă.

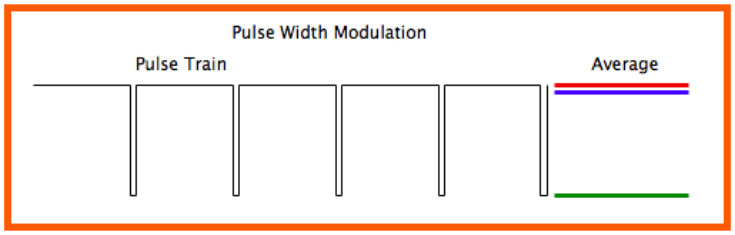
Există calculatoare care pot fi utile pentru determinarea rapidă a valorilor pentru registrele de configurare ale unui Timer:

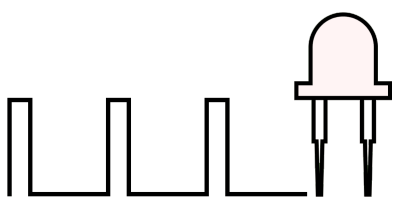
Calculatorul indică valoarea corespunzătoare intervalului de numărare. Pentru a configura valoarea de prag a timer-ului, din această valoare se scade o unitate.

2. PWM (Pulse-width Modulation)

PWM (**P**ulse **W**idth **M**odulation) este o tehnică folosită pentru a varia în mod controlat tensiunea dată unui dispozitiv electronic. Având la dispoziție doar două niveluri de tensiune (0V și 5V), putem aproxima alte valori comutând foarte rapid între cele două niveluri.

Tensiunea primită de un dispozitiv reprezintă media acestor pulsuri și este raportul dintre perioada de timp corespunzătoare valorii ON și perioada totală dintr-un ciclu ON-OFF. Aceast raport se numește **factor de umplere (*duty cycle*)**. Astfel, se pot controla circuite analogice din domeniul digital. Practic, asta înseamnă că un LED acționat astfel se va putea aprinde / stinge gradual, iar în cazul unui motor acesta se va învârti mai repede sau mai încet.



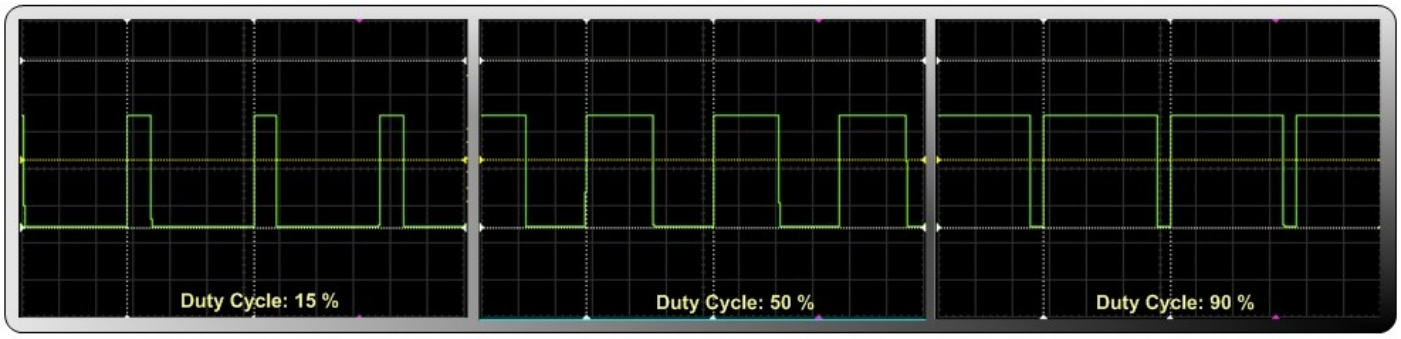


2.1. Principiul de funcționare

Factorul de umplere se exprimă în procente și reprezintă cât la sută din perioada unui semnal acesta va fi pe nivelul ON. În figura de mai jos, se pot observa semnale PWM cu factori de umplere diferiți. Astfel, se poate deduce foarte ușor formula pentru a obține valoarea factorului de umplere (D):

$$D[\%] = \frac{t_on}{t_on + t_off} \cdot 100 = \frac{pulse_width}{period} \cdot 100$$

Astfel, tensiunea medie care ajunge la dispozitiv este dată de relația: D * Vcc.



Modularea folosește variația factorului de umplere a unui semnal dreptunghiular pentru a genera la ieșire o tensiune analogică. Considerând o formă de undă dreptunghiulară *f(t)* cu o valoare minimă *ymin=0* și o valoare maximă *ymax* și factorul de umplere *D* (ca în figură) valoarea medie a formei de undă e dată de relația:

$$\bar{y} = D \cdot ymax$$

Multe circuite digitale pot genera semnale PWM. Majoritatea microcontroller-elor oferă această facilitate, pe care o implementează folosind un numărător care este incrementat periodic (conectat direct sau indirect la o unitate de ceas) și care este resetat la sfârșitul fiecărei perioade a PWM-ului. Când valoarea numărătorului este mai mare decât valoarea de referință, ieșirea PWM (output-ul) trece din starea HIGH în starea LOW (sau invers).

Semnalul PWM poate fi generat și software prin mai multe metode: bit-banging, timer cu ISR. Practic orice metodă prin care se poate genera software secvența:

- Set Pin High
- Wait for T_on
- Set Pin Low
- Wait For T_off

Totuși, în multe situații se dorește o frecvență de ordinul kHz sau zeci de kHz a semnalului PWM (ex. controlul motoarelor DC sau Brushless DC), iar o astfel de metodă nu este cea mai eficientă, fiind necesară intervenția procesorului pentru tratarea întreruperilor frecvente.

2.2. Lucrul cu PWM-ul

Lucrul cu PWM-ul presupune inițializarea unui timer și configurarea output-ului pe pini. Fiecare timer are doi pini pe care poate genera ca output un astfel de semnal (cele două canale): Timer0 are `OC0A` și `0C0B`, Timer1 are `OC1A` și `OC1B` etc.

Atmega328p dispune de 6 canale de PWM distribuite astfel:

- **Timer0 - OCR0A, OCR0B** respectiv **Timer2 - OCR2A, OCR2B** - în total 4 canale pe 8 biți
- **Timer1 - OCR1A, OCR1B** în total 2 canale pe 16 biți

Controller-ul dispune de 6 canale de PWM în cazul în care acestea sunt configurate în moduri ce au ca TOP valorile 0xFF respectiv 0xFFFF. Alternativ, putem configura fiecare timer (atât cele de 8 biți cât și cele de 16 biți) în moduri ce au ca TOP o valoare specificată într-unul din registrele OCRnA sau OCRnB. Un timer configurat astfel are doar un canal ce genereaza output așa cum ne-am aștepta. Și canalul corespunzător registrului ce conține valoare TOP va genera semnal, însa nu în modul dorit.

Pe Atmega328p există 3 tipuri de PWM care pot fi generate:

- Fast PWM
- Phase Correct PWM
- Phase and Frequency Correct PWM

Ne vom ocupa în principal de modul Fast PWM care este utilizat pentru majoritatea aplicațiilor, mai puțin cele în care este nevoie de un control precis (de exemplu motoare BLDC, audio).

Fast PWM

Numărarea se face doar pe frontul crescător al semnalului de ceas. În modul Fast PWM, modificarea factorului de umplere se realizează instant, în schimb semnalul nu este centrat (este defazat, practic apare un "glitch" la modificarea semnificativă a factorului de umplere). Există mai multe moduri de Fast PWM oferite de către microcontroller. De exemplu, pentru Timer-ul 1 avem:

- Fast PWM pe 8 biți, cu valoarea de TOP = 0x00FF.
- Fast PWM pe 9 biți, cu valoarea de TOP = 0x01FF.
- Fast PWM pe 10 biți, cu valoarea de TOP = 0x03FF.
- Fast PWM cu valoarea de TOP în **ICR**
- Fast PWM cu valoarea de TOP în **OCRnA**

Frecvența semnalului PWM depinde de prescaler și de frecvența oscilatorului. Din secțiunea 15.9.3 (pag 102), formula de calcul a frecvenței în modul Fast PWM 8-bit este:

$$f_{OCnX} = \frac{f_{clk}}{N \cdot (TOP + 1)} = \frac{f_{clk}}{N \cdot 256}$$

În acest laborator vom folosi doar modul Fast PWM. Pentru variantele de Fast PWM oferite de către celelalte timere (0 și 2) și pentru celelalte moduri PWM, consultați datasheet-ul.

De exemplu, presupunem că avem Timer1 configurat pe modul de funcționare Fast PWM (modul de funcționare nu trebuie neapărat să conțină cuvântul 'PWM' ca să poată fi folosit pentru generarea unui semnal). Fast PWM este caracterizat de o frecvență fixă și un prag stabilit de programator, ce poate fi modificat în timpul execuției.

- Modul `1 0` pentru biții `COM1A1 COM1A0` va lăsa semnalul de pe pinul `OC1A` pe 1 în timpul numărătorii (până la atingerea pragului) și va pune semnalul pe 0 de la atingerea pragului până la capătul unui ciclu (numărare completă până la TOP).
- Pentru a obține un factor de umplere x%, aici vom seta `OCR1A = x * TOP / 100` (pentru Fast PWM, TOP poate fi 0xFF, 0x1FF sau 0x3FF, în funcție de configurația aleasă)

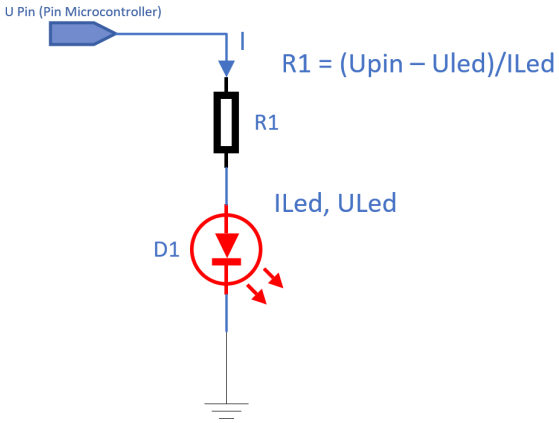
Exemplu de inițializare a Timer1 in modul Fast PWM 8-bit non-inverting, cu Prescaler la valoarea 1024:

```
CCR1A = 0;
TCCR1B = 0;
TCNT1  = 0;
//PB1 output - OC1A este PB1
DDRB |= (1 << PB1);
//pentru modul Fast PWM 8-bit, biții WGMn0 si WGMn2 au valoarea 1
TCCR1A |= (1 << WGM10);
//TCCR1A conține doar biții WGM10 si WGM11, WGM12 și WGM13 se găsesc in TCCR1B
TCCR1B |= (1 << WGM12);
//pentru modul non-inverting, COM1A1 = 1 și COM1A0 = 0
TCCR1A |= (1 << COM1A1);
```

```
//pentru Prescaler de 1024 scriem 1 pe CS12 si CS10
TCCR1B |= (1 << CS12) | (1 << CS10);
//Pragul la care se schimbă semnalul pentru a obține un factor de umplere de 0.5
//Deoarece în acest mod TOP este 0xFF, OCR1A va fi 50 * 255 / 100 = 127
OCR1A = 127;
sei();
```

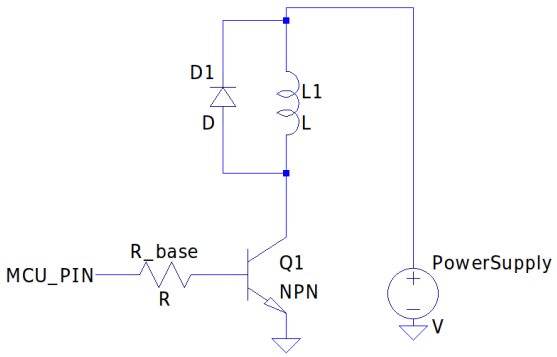
3. Aplicații cu PWM în Arduino

Pentru a controla un LED folosind un semnal PWM, se poate conecta la fel ca în cazul on/off, folosind o rezistență de limitare a curentului. Dacă avem un LED de putere (ex. LED-uri auto, LED-uri pentru iluminare) este necesar un driver specializat.



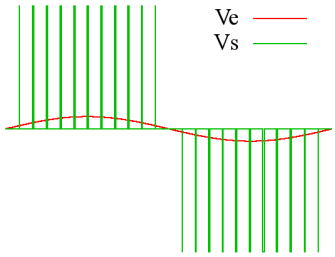
În cazul unui motor, căruia i se aplică un semnal PWM cu factor de umplere de 0%, viteza de rotație a acestuia va fi egală cu 0 RPM. Un factor de umplere de 100% va duce la o turație maximă a acestuia. Pentru astfel de cazuri în care avem sarcină inductivă și curenți mari (releuri, motoare, inductoare, electrovalve, etc.) este necesară utilizarea unui driver (element de comutație/tranzistor, driver motor, etc.)

Pentru astfel de cazuri în care avem sarcină inductivă și curenți mari, este necesară și o diodă flyback (Wikipedia Flyback diode [https://en.wikipedia.org/wiki/Flyback_diode]). Semnalul PWM generat de microcontroller va comanda driver-ul (ex. baza tranzistorului printr-o rezistență) iar dioda flyback va prelua spike-urile de tensiune generate de motor/sarcina inductivă la închiderea tranzistorului (care altfel pot duce la distrugerea tranzistorului).



O altă utilizare a semnalelor PWM este de a genera semnal analogic cu aplicații în amplificare audio (de clasă D), încărcătoare de baterii, etc. Prin modificarea periodică a factorului de umplere, se pot obține semnale ce aproximează un semnal analogic (ex. o sinusoidă).

Generarea unui semnal sinusoidal folosind PWM:



Semnalele PWM pot avea și alte utilizări, precum trimiterea de comenzi către un dispozitiv de acționare. Un astfel de exemplu este servomotorul, care primește comenzi determinate de factorul de umplere al semnalului PWM pentru a controla poziția. Servo control [https://en.wikipedia.org/wiki/Servo_control]

În continuare, ne vom folosi de biblioteca Arduino pentru a realiza niște aplicații ale semnalelor PWM: controlul unui LED RGB și controlul poziției unui servomotor hobby. Alternativ, pentru cei care doresc să aprofundeze modul de programare la nivel de registre, se pot realiza aplicațiile folosind timere în modul Fast PWM și/sau CTC în loc de bibliotecile din Arduino (*analogWrite*, *Servo*).

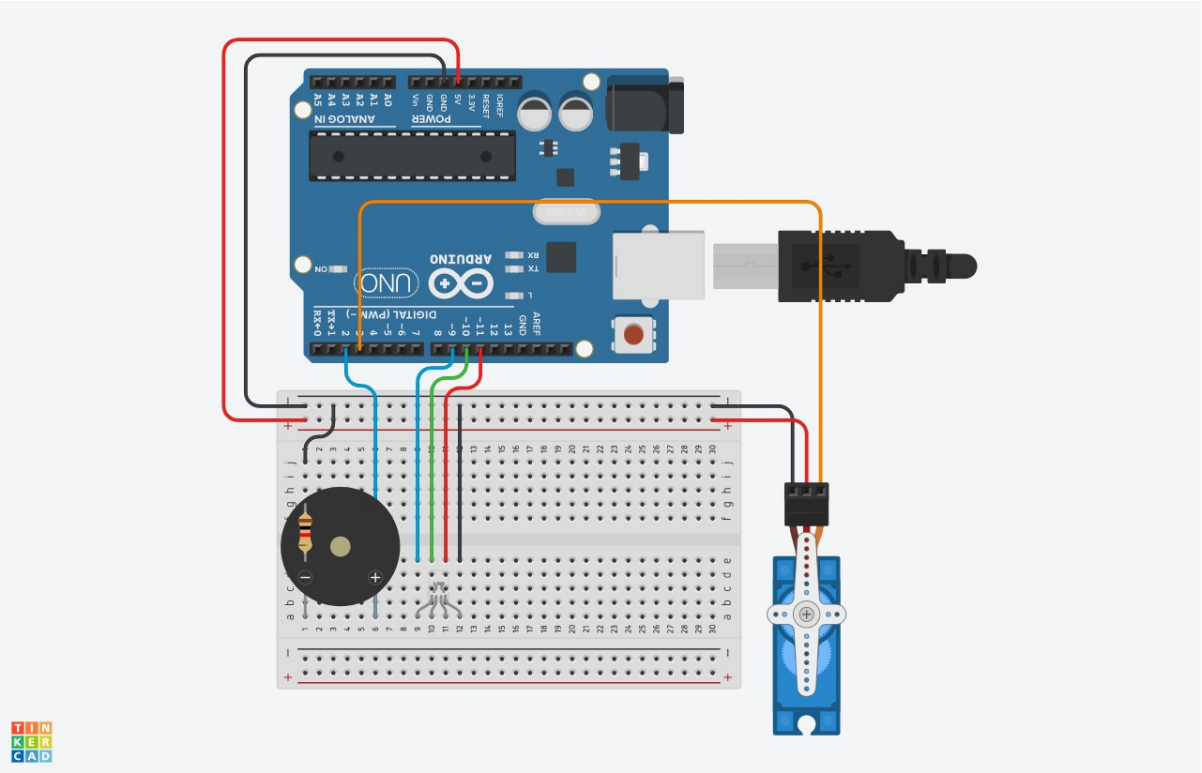
AnalogWrite

Funcția *analogWrite* din Arduino, configurează de fapt un timer în modul **Fast PWM pe 8 biți** și poate genera semnal PWM (doar) pe pinii asociați unuia dintre timere. `analogWrite(pin_arduino, value_0_255)` De exemplu, pe Arduino/Atmega328p avem următorii pini care pot genera semnal PWM folosind funcția *analogWrite*:

Pin Arduino	Pin Atmega328p	Timer output	Frecvența PWM (implicit)
5	PD5	OC0B (Timer0)	980 Hz
6	PD6	OC0A (Timer0)	980 Hz
9	PB1	OC1A (Timer1)	490 Hz
10	PB2	OC1B (Timer1)	490 Hz
11	PB3	OC2A (Timer2)	490 Hz
3	PD3	OC2B (Timer2)	490 Hz

Este posibil să avem nevoie să modificăm frecvența semnalului PWM, caz în care trebuie să configurăm explicit Timer-ul (ex. prescaler, TOP) folosind registre. SecretsOfArduinoPWM [https://www.arduino.cc/en/pmwiki.php?n=Tutorial/SecretsOfArduinoPWM]

4. Exerciții



4.1. Task 0

Folosind Timer 1 de pe Arduino și operațiile cu regiștri, afișați pe interfața serială un mesaj la alegerea voastră o dată la 5 secunde.

Timerul de pe Arduino nu poate cronometra perioade atât de lungi. Veți fi nevoiți să afișați mesajul după un anumit număr de perioade, de lungime aleasă de voi. Ex: perioadă de 0.5s (2Hz), afișăm mesajul la fiecare a doua intrare în funcția asociată întreruperii de timer; perioadă de 0.1s (10Hz), afișăm mesajul la fiecare a 10-a intrare.

Alegerea parametrilor pentru timer pentru setările registrelor:

```
OCR1A = 31249; // compare match register 16MHz/256/2Hz-1 => 2Hz
TCCR1B |= (1 << WGM12);   // CTC mode
TCCR1B |= (1 << CS12);    // 256 prescaler
TIMSK1 |= (1 << OCIE1A);  // enable timer compare interrupt
```

4.2. Task 1

Folosind codul de la exercițiul anterior, la fiecare 5 secunde veți alterna între două stări de redare a unei melodii pe buzzer-ul din montaj:

- viteză normală și octavă normală
- viteză dublă (noteDuration / 2) și octavă superioară

Creșterea unei octave se realizează dublând frecvența notelor. Pentru redarea unei note veți folosi funcția tone() [https://www.arduino.cc/reference/en/language/functions/advanced-io/tone/]. Urmăriți exemplele de pe acest repository arduino-songs [https://github.com/robsoncouto/arduino-songs] și alegeți o melodie.

Realizați redarea melodiei în funcția loop() pentru a putea folosi întreruperile generate de timer și pentru a reda continuu melodia.

4.3. Task 2

Folosind Timer 0, ciclați led-ul RGB între mai multe culori și mișcați servo-motorul câte 10° la fiecare x secunde (alegeți voi o perioadă potrivită pentru urmărirea schimbărilor). Nu uitați de sugestia de la primul task (va trebui să numărați un număr de perioade mult mai scurte decât cea dorită).

Atenție la limitele de unghi ale servo-motorului. Depășirea acestora poate avaria servomotorul. Verificați ca unghiul să nu depășească intervalul [0°, 180°] (sau [0, pi] pentru cei ce își amintesc cu plăcere de trigonometrie).

La acest task veți afișa diverse culori cu ajutorul PWM-ului și veți controla un servo-motor:

- Cu funcția analogWrite [https://www.arduino.cc/reference/en/language/functions/analog-io/analogwrite/] veți seta factorul de umplere PWM pentru fiecare culoare
- Exemplu pentru controlul unui servo-motor ce oscilează în intervalul [0°, 180°]

```
#include <Servo.h>

Servo myservo;  // create servo object to control a servo
// twelve servo objects can be created on most boards

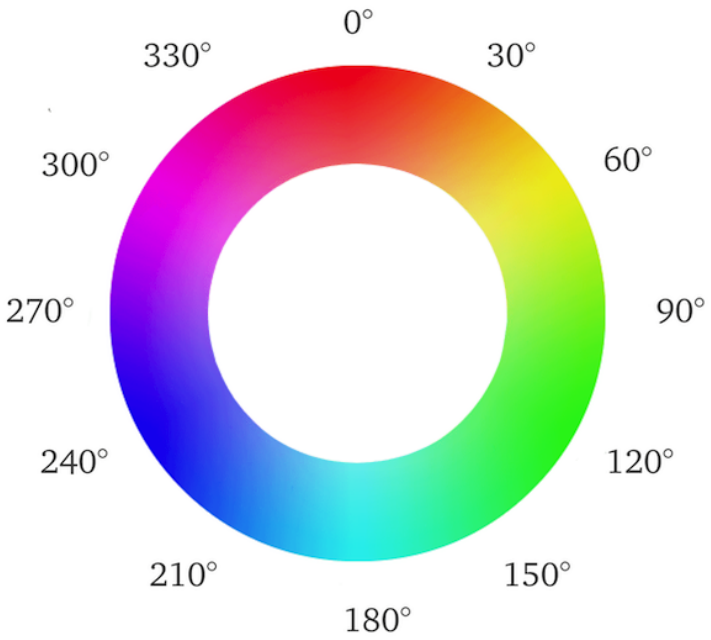
int pos = 0;    // variable to store the servo position

void setup() {
  myservo.attach(3);  // attaches the servo on pin 3 to the servo object
  // test led
  DDRD |= (1 << PD7);
  PORTD &= ~(1 << PD7);
}

void loop() {
  for (pos = 0; pos <= 180; pos += 1) { // goes from 0 degrees to 180 degrees
    // in steps of 1 degree
    myservo.write(pos);              // tell servo to go to position in variable 'pos'
    delay(15);                       // waits 15ms for the servo to reach the position
  }
  for (pos = 180; pos >= 0; pos -= 1) { // goes from 180 degrees to 0 degrees
    myservo.write(pos);              // tell servo to go to position in variable 'pos'
    delay(15);                       // waits 15ms for the servo to reach the position
  }
}
```

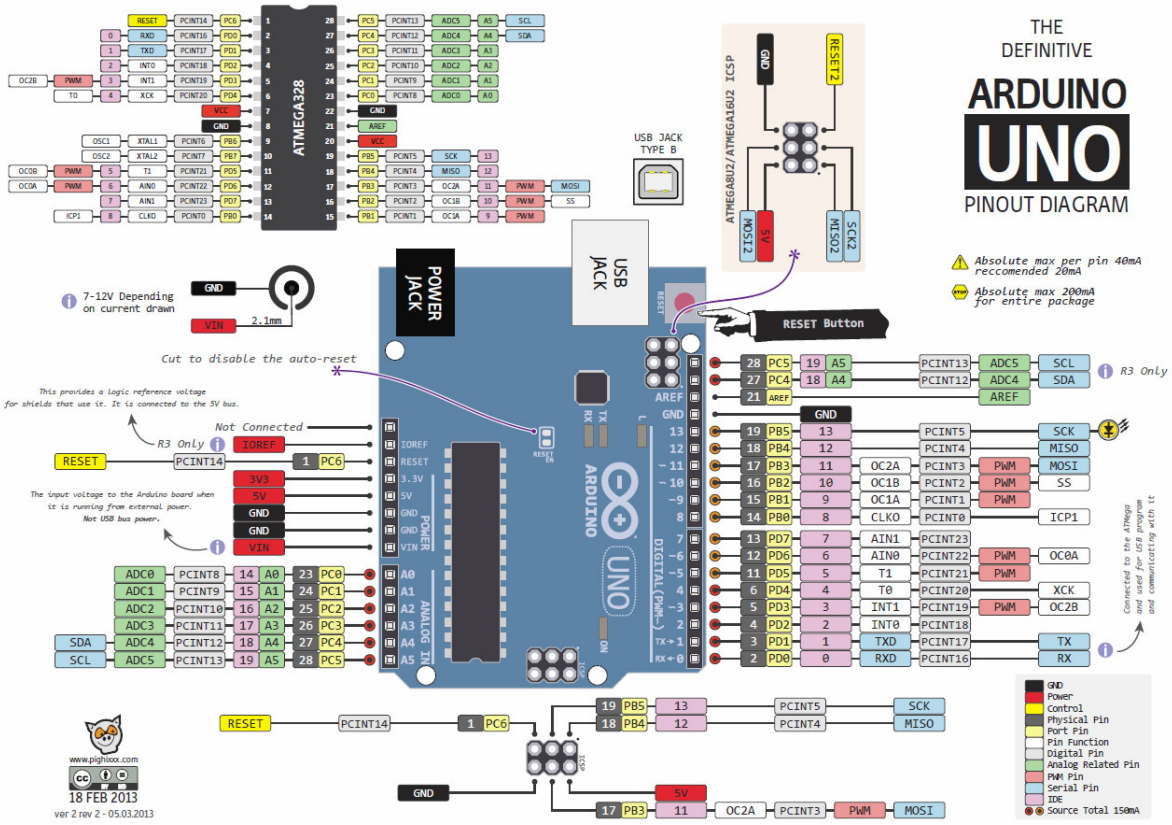
Bonus : modificați programul folosind functia *setLedColorHSV()*

- Funcția *setLedColorHSV* permite modificarea culorii folosind reprezentarea alternativă HSV (Hue Saturation Value), fiind mai ușor apoi de modificat culoarea, saturația și intensitatea luminoasă. setLedColorHSV
- Setați valorile pentru saturație (s) și intensitate (v) pe 1 și modificați culoarea (h) în intervalul 0-360
- Pentru a urmări corespondența dintre cele 2 reprezentări (RGB și HSV) există selectoare de culori precum Online Color Picker [https://colorpicker.me/#3237b9]



5. Resurse

- Datasheet Atmega 328p
- Arduino UNO pinout



Responsabili:

- Alexandru Predescu [mailto:alexandru.predescu@upb.ro]
- Dragos Ene [mailto:enedragos99@gmail.com]

- Narcis Caroi [mailto:narciscaroi24@gmail.com]

6. Linkuri utile

- AVR Libc - interrupt.h [http://www.nongnu.org/avr-libc/user-manual/group__avr__interrupts.html]
- debouncing [https://www.arduino.cc/en/Tutorial/BuiltInExamples/Debounce]
- SecretsOfArduinoPWM [https://www.arduino.cc/en/pmwiki.php?n=Tutorial/SecretsOfArduinoPWM]
- Servo control [https://en.wikipedia.org/wiki/Servo_control]

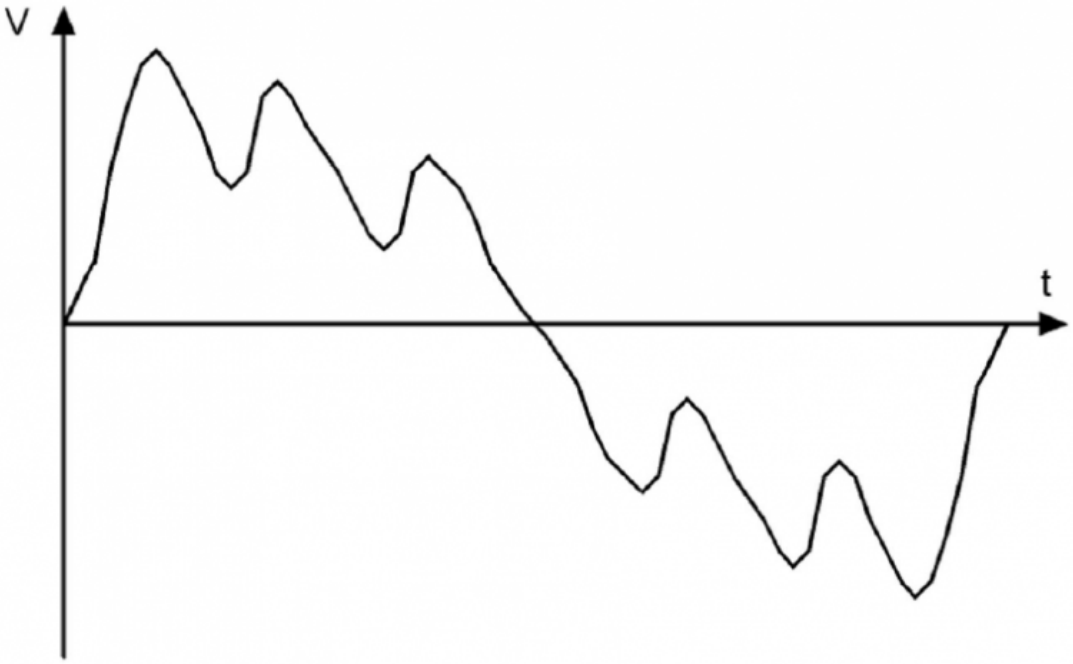
pm/lab/lab3-2023.txt · Last modified: 2023/03/27 11:26 by narcis_florin.caroi

Laborator 4: Analog Digital Convertor (ADC)

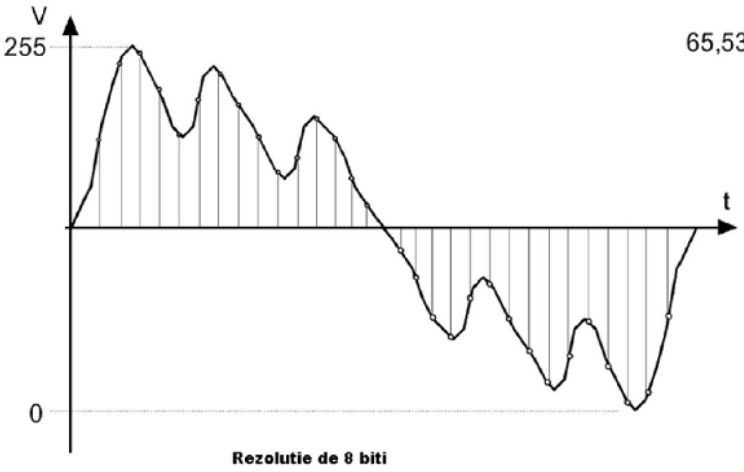
Acest laborator are ca scop familiarizarea voastră cu lucrul cu convertorul analog-digital prezent în microcontroller-ul Atmega328p.

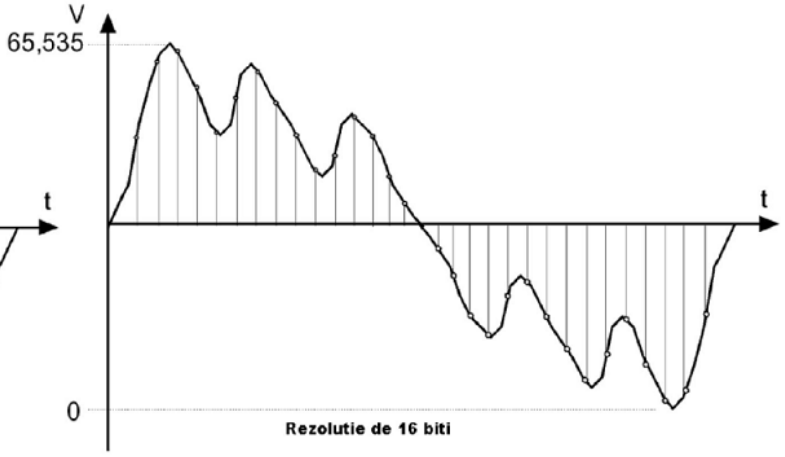
1. Măsurarea semnalelor analogice

Pentru a putea măsura semnalele analogice într-un sistem de calcul digital, acestea trebuie convertite în valori numerice discrete. Un convertor analog – digital (en. Analog to Digital Converter - ADC) este un circuit electronic care convertește o tensiune analogică de la intrare într-o valoare digitală.



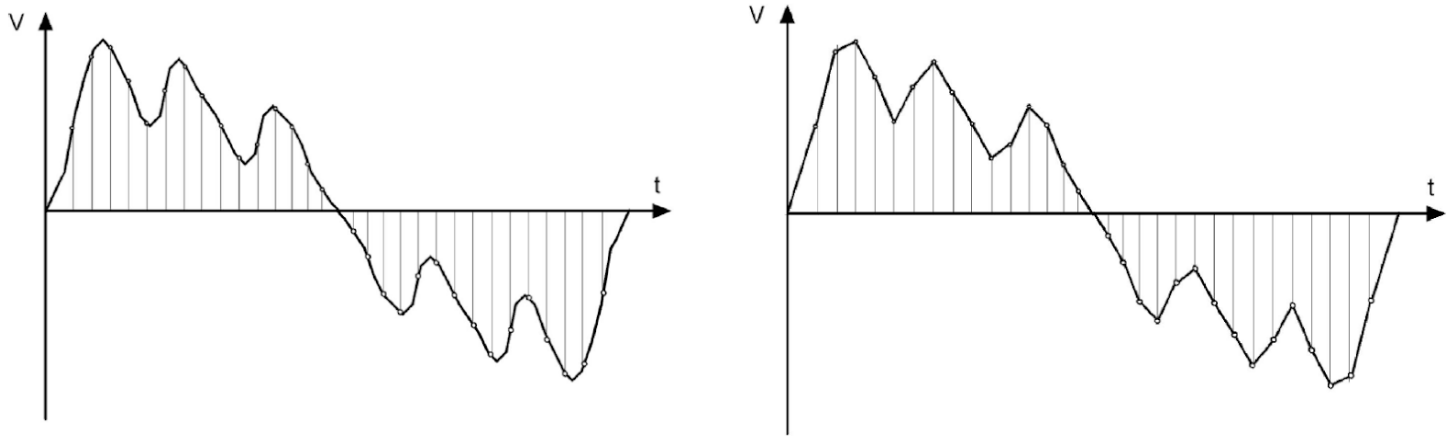
O caracteristică importantă a unui ADC o constituie **rezoluția** acestuia. Rezoluția indică numărul de biți pe care convertorul poate să reprezinte rezultatul binar. le furnizeze la ieșirea sa în intervalul de măsură. **Cuanta de măsurare** - cea mai mică valoare care poate fi distinsă de ADC reprezintă raportul între domeniul tensiunii de intrare (diferența între tensiunea maximă și tensiunea minimă ce pot fi aplicate la intrarea în convertor) și numărul maxim de valori binare reprezentabile (2^N).





De exemplu, dacă rezoluția unui convertor este de 10 biți atunci el poate furniza 2^10 = 1024 valori diferite la ieșire. Dacă gama de măsurare este de la 0 la 5V, cuanta de măsurare va fi: (5V-0V)/1024 = 0.0048V adică 4.8mV.

O altă caracteristică importantă a unui convertor analog-digital o constituie rata de eșantionare (e.n. sampling rate). Aceasta depinde de timpul dintre două conversii succesive și afectează modul în care forma de undă originală va fi redată după procesarea digitală. Mai jos observăm cum arată modul în care semnalul eșantionat va fi reconstituit în urma trecerii printr-un convertor digital – analog (DAC). După cum se poate observa, semnalul reprodus nu este identic cu cel original. Dacă rata de eșantionare ar crește semnalul reprodus prin unirea punctelor/eșantioanelor digitale aproximează din ce în ce mai bine originalul.



Care este însa rata minimă de eșantionare pentru a reproduce fără pierderi un semnal de o frecvența data? **Teorema lui Nyquist** spune că o rată de eșantionare de **minimum două ori mai mare decât frecvența semnalului măsurat** este necesară pentru acest lucru, teorema aplicându-se și pentru un semnal compus dintr-un intreg spectru de frecvențe, cum ar fi vocea umană sau o melodie. Limitele maxime ale auzului uman sunt 20Hz – 20kHz dar frecvențele obișnuite pentru voce sunt în gama 20-4000Hz, de aceea centralele telefonice folosesc o rată de eșantionare a semnalului de 8000Hz. Rezultatul este o reproducere inteligibilă a vocii umane, suficientă pentru transmiterea de informații într-o convorbire obișnuită. Pentru reproducerea fidelă a spectrului audibil se recurge la rate mai mari de eșantionare. De exemplu, înregistrarea pe un CD are o rată de eșantionare de 44100Hz ceea ce este mai mult decât suficient pentru reproducerea fidelă a tuturor frecvențelor audibile.

In funcție de modul în care se execută conversia, convertoarele analog-digitale pot fi de mai multe tipuri:

- ADC paralel (Flash)
- ADC cu aproximare succesivă
- ADC cu integrare (single-slope, dual-slope);
- ADC Sigma-delta (delta-sigma, 1-bit ADC sau ADC cu oversampling).

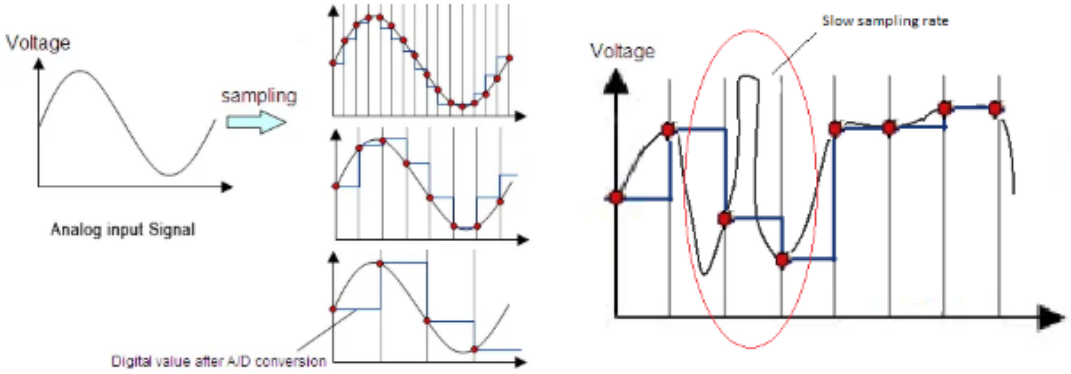
Resolution

The resolution represents the number of discrete values that can be produced over the range of analog values in a measurement period. The discrete values are usually in binary form, so a common way to represent resolution is in audio bit depth (the number of bits of information in each sample). For example, for a resolution of 8 bits, we can encode the analog input signal in 256 different levels, the values ranging from 0 to 255. Resolution can also be expressed in volts, each level representing a fraction of the voltage measurement range. For example, for a range of 0- 5V and an 8-bit resolution, the subdivision would be (5V – 0V) / (2 ^ 8).

Sampling Rate

The sampling rate represents the time at which the values of the analog input signal are read. The time is divided into discrete intervals, each sampling a value from the input signal. The Nyquist frequency tells us that it is mandatory for the sampling rate to be at least twice the frequency of the analog input signal so that the signal can be reconstructed afterwards. There appears a problem where frequencies above half the Nyquist frequency are

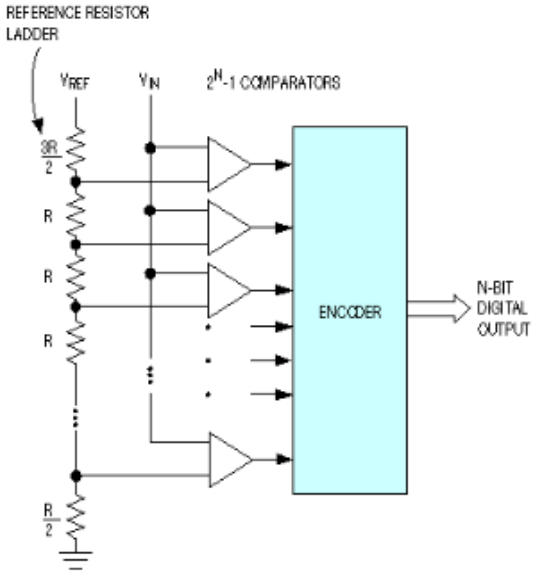
sampled because they are incorrectly detected as lower frequencies. This process is called aliasing.



ADC Types

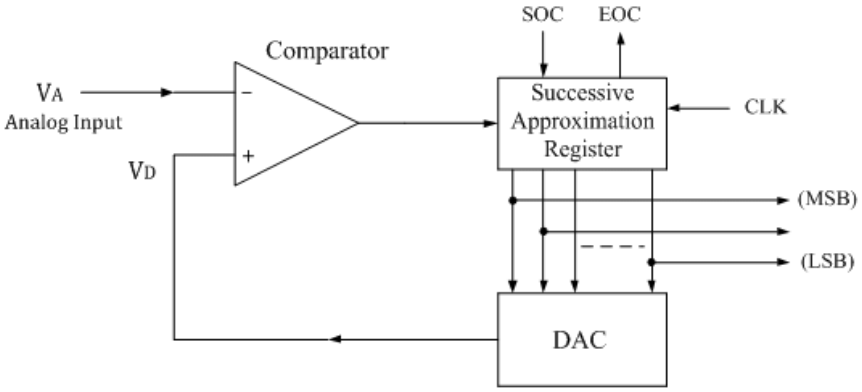
Direct Conversion (Flash ADC)

Has a bank of comparators sampling the input signal in parallel, each firing for their decoded voltage range. The comparator bank feeds a logic circuit that generates a code for each voltage range.



Successive Approximation

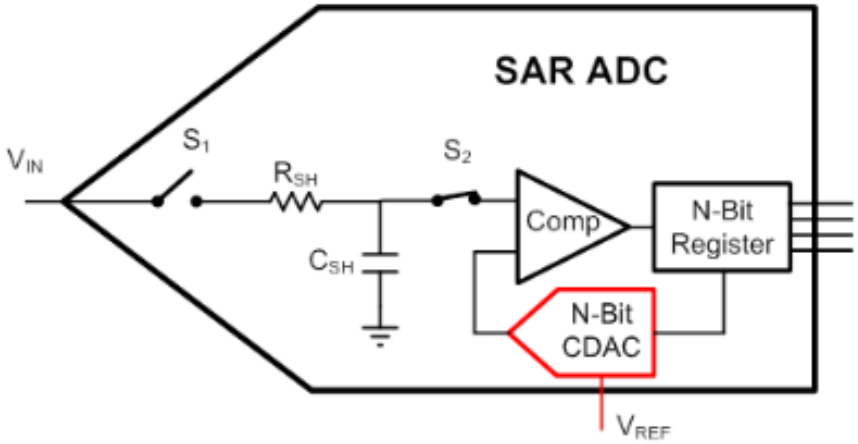
Uses a comparator to successively narrow a range that contains the input voltage. At each successive step, the converter compares the input voltage to the output of an internal digital to analog converter which might represent the midpoint of a selected voltage range. At each step in this process, the approximation is stored in a successive approximation register (SAR).



Successive approximation ADC`s are cheaper than direct conversion ADC`s, and have a smaller footprint on the IC, but they are also slower.

Sigma-Delta

Is a newer design that offers low conversion noise and good resolution. Sigma-delta convertors are appropriate for application that require high resolution, but not the fastest conversion. As stated above, an ADC conversion is not instant, it requires a certain amount of time to complete. This raises a issue: what will happen with the result of the conversion if the input signal will change during this conversion time. The conversion result might not correspond then to the real value of the signal. To solve this, an internal input buffer is used.



When the input signal is sampled, switch S1 is closed, S2 open, so that the Csh capacitor will hold a voltage equal to the input signal. S1 will then open, S2 will close and the conversion will begin. The two switches will hold this state until the conversion will end. Capacitor Csh will hold the (analog) value of the signal while the during the A/D conversion.

2. Convertorul ADC al Atmega 328p

Convertorul analog-digital inclus în microcontroller-ul Atmega328p este un ADC cu aproximări succesive. Are o rezoluție de pana la 10 biți și poate măsura orice tensiune din gama 0-5V de pe opt intrări analogice multiplexate (6 disponibile pe arduino).

Acest convertor poate fi controlat prin doua registre de stare și control (ADCSRA si ADCSRB) și un registru cu biți de selecție pentru multiplexoare (ADMUX). În primele, putem stabili când să se efectueze conversia, dacă să se genereze întrerupere la finalul unei conversii etc. Folosind registrul pentru multiplexoare se alege ce canal va genera input pentru convertor si tensiunea de referință. De asemenea, în afară de aceste două registre mai avem registrul ADC în care este scris rezultatul conversiei (ADC).

Relația dintre valoarea din registrul ADC și tensiunea măsurată este următoarea:

ADC = V_in * 1024 / V_ref

sau

V_in = ADC * V_ref / 1024

unde *V_in* este tensiunea măsurată iar *V_ref* este tensiunea aleasă ca referință.

Tensiune de referinta

In functie de intervalul in care varieaza semnalul pe care il citim putem selecta alta tensiune de referinta. Acest lucru este util pentru a mari rezolutia citirii. ADC-ul de pe Atmega 328p ne ofera ca tensiune de referinta tensiunea de alimentare (AVCC), o tensiune interna de 1.1V sau un pin pe care putem conecta o referinta de tensiune externa (AREF)

Prescaler

ADC-ul nostru are nevoie de un semnal de ceas pentru a stii cat timp dureaza o conversie. Pentru ca semnalul de ceas al microcontrollerului este prea rapid avem nevoie de un prescaler. Cea mai mica valoare este 2 si cea mai mare este 128.

F_ADC = F_CPU / PRESCALER

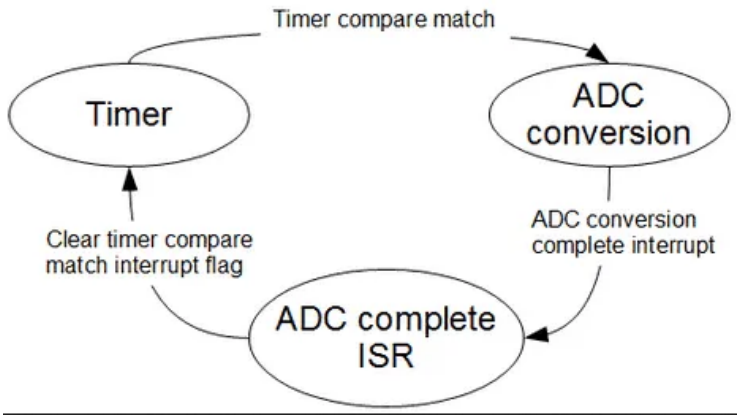
Alegerea prescaler-ului depinde de frecventa de esentionare si de acuratetea dorita. Cu cat prescaler-ul este mai mare frecventa ADC va fi mai mica si acuratetea va fi mai mare. Mai multe informatii se pot gasi in capitolul 23.4 din datasheet.

Moduri de functionare

Convertorul poate functiona in mai multe moduri. Cel mai comun mod si cel care este desponibil si in biblioteca arduino este **Single Conversion Mode**. In acest mod se va efectua o singura conversie si se incepe setand ADSC pe 1. Registrul va fi setat pe 0 automat la finalul conversiei.

Un alt mod de functionare este **Free Running Mode** in care convertorul va functiona in continuu, la finalul unei conversii se va incepe automat urmatoarea conversie. De fiecare data rezultatul precedent va fi suprascris. Pentru a incepe o conversie se va seta registrul ADSC pe 1 si acesta nu va fi pus pe 0 automat.

- **External Interrupt Request**. In acest mod o conversie va incepe pe un front pozitiv al unui pin de intrare.
- **Analog Compare mode**. In acest mod ADC-ul poate compara doua semnale analogice. Nu vom folosi aceasta functionalitate in cadrul laboratorului. Mai multe detalii in capitolul 23 din datasheet.
- **Timer mode**. Pentru un control mai precis al momentului in care se intampla conversia, se poate folosi un eveniment de timer pentru a incepe o noua conversie.



Registre

ADMUX - ADC Multiplexer Selection Register

Bit	7	6	5	4	3	2	1	0	
(0x7C)	REFS1	REFS0	ADLAR	–	MUX3	MUX2	MUX1	MUX0	ADMUX
Read/Write	R/W	R/W	R/W	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Bit 7:6 - **REFS1:0: Reference Selection Bits** → Selecteaza tensiunea de referinta
- Bit 5 - **ADLAR ADC: Left Adjust Result** → Modul de aliniere a celor 10 biti. (left-aligned sau right-aligned)
- Bits 3:0 - **MUX3:0: Analog Channel Selection Bits** → Selecteaza portul de intrare de pe care se face conversia

ADCSRA – ADC Control and Status Register A

Bit	7	6	5	4	3	2	1	0	
(0x7A)	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – ADEN: ADC Enable** → Porneste convertorul
- **Bit 6 – ADSC: ADC Start Conversion** → Folosit pentru a incepe o conversie

- **Bit 5 – ADATE: ADC Auto Trigger Enable** → Porneste modul in care se conversiile ADC incep automat in functie de o sursa specificata in ADCSRB
- **Bit 4 – ADIF: ADC Interrupt Flag** → Setat pe 1 atunci cand o conversie s-a terminat. Acest flag este sters automat atunci cand se apeleaza o rutina de tratate a intreruperilor.
- **Bit 3 – ADIE: ADC Interrupt Enable** → Porneste intreruperile pentru ADC
- **Bits 2:0 – ADPS2:0: ADC Prescaler Select Bits** → Setare prescaler

ADCSRB – ADC Control and Status Register B

Bit	7	6	5	4	3	2	1	0	
(0x7B)	–	ACME	–	–	–	ADTS2	ADTS1	ADTS0	ADCSRB
Read/Write	R	R/W	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 2:0 – ADTS2:0: ADC Auto Trigger Source** → Sursa de la care sa se genereze o noua conversie. Depinde de ADATE din ADCSRA

Exemplu

Setup:

```
ADMUX = 0;
/* ADC1 - channel 1 */
ADMUX |= (1 << MUX0);
/* AVCC with external capacitor at AREF pin */
ADMUX |= (1 << REFS0);

ADCSRA = 0;
/* set prescaler at 128 */
ADCSRA |= (7 << ADPS0);
/* enable ADC */
ADCSRA |= (1 << ADEN);
```

Read:

```
/* start conversion */
ADCSRA |= (1 << ADSC);
/* wait until conversion is complete */
while (!(ADCSRA & (1 << ADIF)));
uint16_t result = ADC;
```

3. ADC in Arduino

Biblioteca arduino ne pune la dispozitie o functie simpla pentru a folosi ADC-ul si anume **analogRead()**.

```
void loop() {
  val = analogRead(A0);  // read the input pin
  Serial.println(val);   // debug value
  delay(100);
}
```

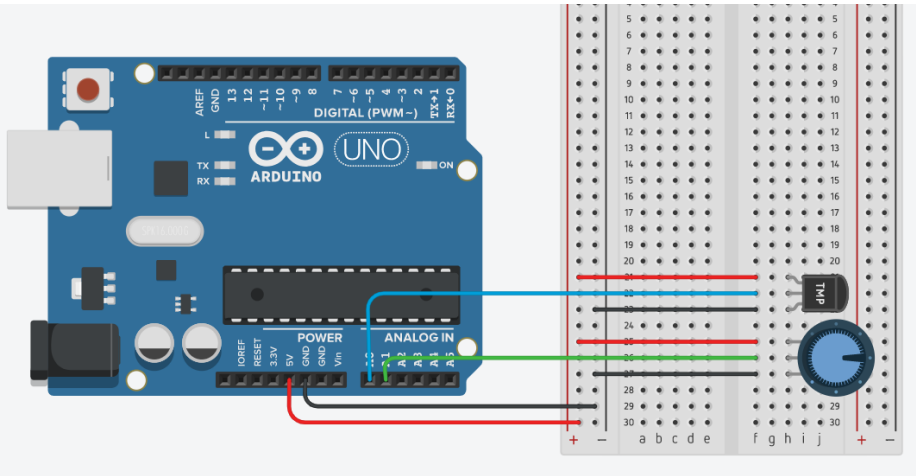
Aceasta functie primeste ca parametru un pin si va bloca pana citirea valorii de pe acel pin se termina. Pentru aplicatii simple acest lucru este suficient, dar pentru aplicatii mai complexe in care ne dorim sa continuam procesarea in timp ce facem conversia va trebui sa folosim cod specific pentru microcontrollerul nostru.

Tensiunea de referinta folosita se poate seta cu functia analogReference()

4. Exercitii

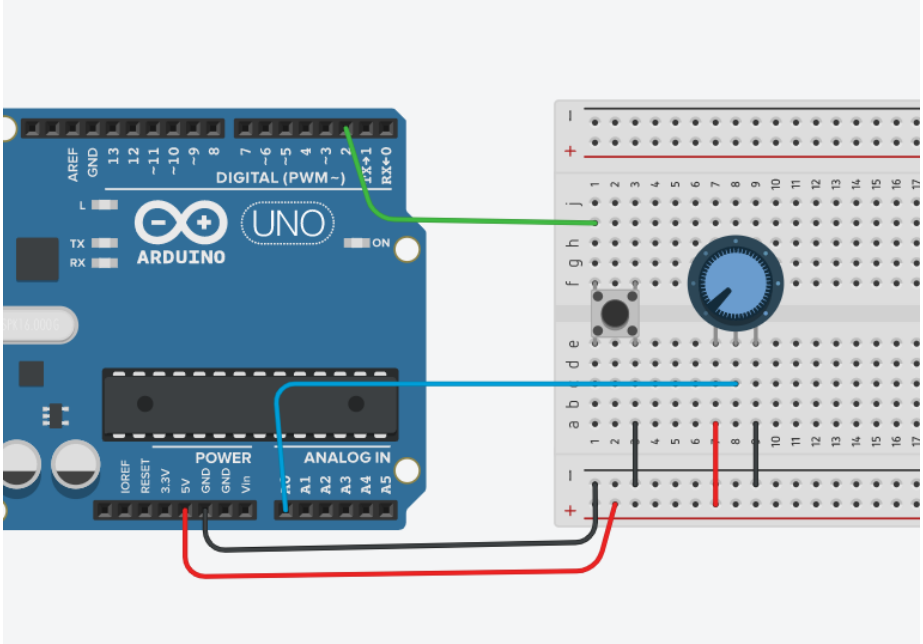
Task 0 Folositi cod Arduino pentru a citi valoarea unui potentiometru si a unui senzor de temperatura si apoi trimiteti valorile pe seriala.

1. Transmiteți către consolă, folosind seriala disponibilă, tensiunea (calculată pe microcontroller) de la ieșirea potențiometrului și valoarea returnată de ADC (0-1023).
2. Valoarea citită pentru senzorul de temperatură trebuie sa fie exprimată in grade Celsius. Tensiunea de iesire a senzorului variaza liniar cu temperatura. Puteti sa faceti conversia experimental sau folosindu-vă de datasheet (Datasheet TMP36 [https://www.analog.com/media/en/technical-documentation/data-sheets/TMP35_36_37.pdf] / Datasheet LM35 [https://www.ti.com/lit/ds/symlink/lm35.pdf]).



Rezolvare

Task 1 Folosind cod specific AVR, cititi valoarea potentiometrului doar atunci cand se apasa un buton.



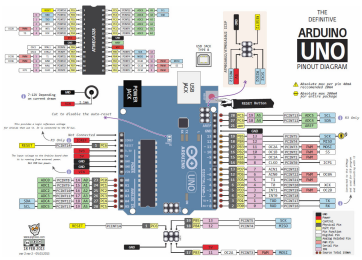
Rezolvare

Task 2 Folosind cod specific AVR, cititi valoarea potentiometrului o data la 10ms. Pentru acest exercitiu trebuie sa folositi ADC-ul sa porneasca o conversie automat in functie de un timer.

Rezolvare

5. Resurse

- Datasheet Atmega 328p
- Arduino UNO pinout



- Responsabili: Florin Stancu [mailto:florin.stancu@upb.ro]

Rezolvare

6. Linkuri Utile

- Arduino Analog Read [https://www.arduino.cc/reference/en/language/functions/analog-io/analogread/]
- Datasheet TMP36 [https://www.analog.com/media/en/technical-documentation/data-sheets/TMP35_36_37.pdf]
- Datasheet LM35 [https://www.ti.com/lit/ds/symlink/lm35.pdf]
- Datasheet ATmega328p [https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf]
- Tinkercad TMP36 [https://www.tinkercad.com/things/cl7vDLkEZwF?sharecode=Ve7hoj8NLhZ_JSmV2jcoruc8lzp35D1E-pg1nTFhrgE]

pm/lab/lab4-2022.txt · Last modified: 2023/04/04 08:02 by florin.stancu

Laboratorul 5: SPI (Serial Peripheral Interface)

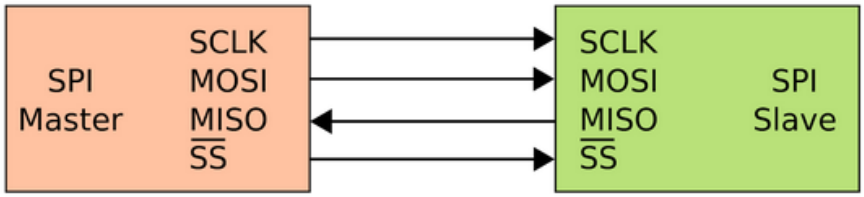
Acest laborator acoperă noțiunea de SPI. Pentru aprofundarea acestui topic, consultați Datasheet ATmega328P [https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf] și Serial Peripheral Interface.

1. SPI (Serial Peripheral Interface)

SPI este un standard sincron dezvoltat de Motorola care funcționează în modul full-duplex (transferul de date are loc în ambele direcții simultan). Dispozitivele comunică utilizând o arhitectură Master-Slave (este permis un singur dispozitiv Master și unul sau mai multe dispozitive Slave). Dispozitivul Master este cel care inițiază comunicarea. SPI mai este numit și „four wire”. Cele patru semnale utilizate sunt următoarele:

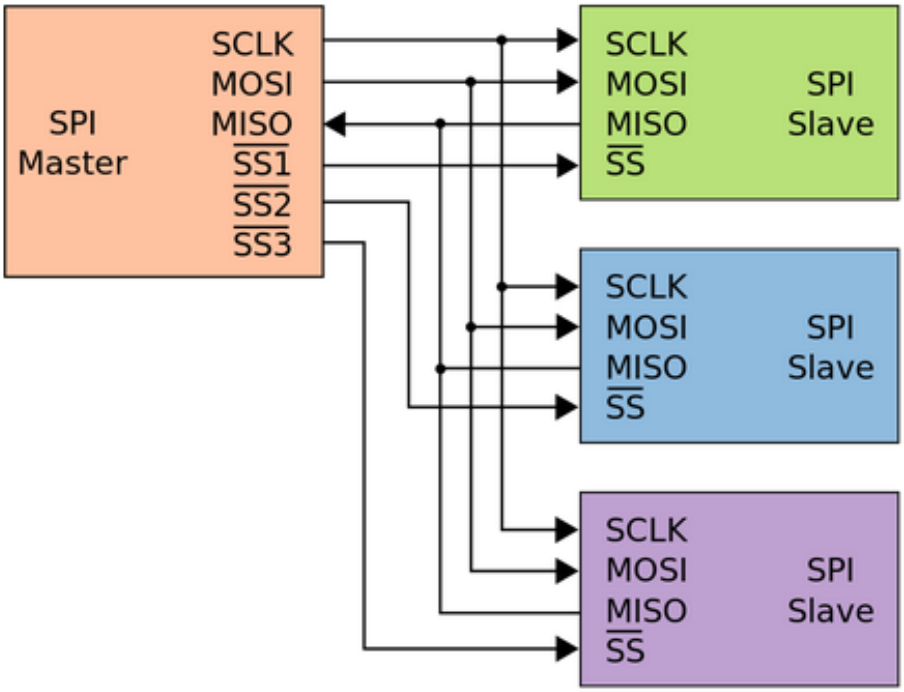
- MOSI — Master Output Slave Input (transmiterea datelor de la Master la Slave)
- MISO — Master Input Slave Output (transmiterea datelor de la Slave la Master)
- SCLK — Serial Clock (sincronizarea dintre dispozitive. Controlat de Master)
- CS/SS — Chip Select/Slave Select (selectarea dispozitivului Slave de către Master. Valoarea LOW pentru Slave-ul selectat)

Putem vedea că există două semnale utilizate pentru transmiterea datelor: MOSI și MISO. Transmiterea datelor pe SPI se face sincron folosind semnalul de ceas SCLK. Semnalul CS / SS este utilizat pentru a selecta dispozitivul slave pe magistrala care este adresată în prezent.



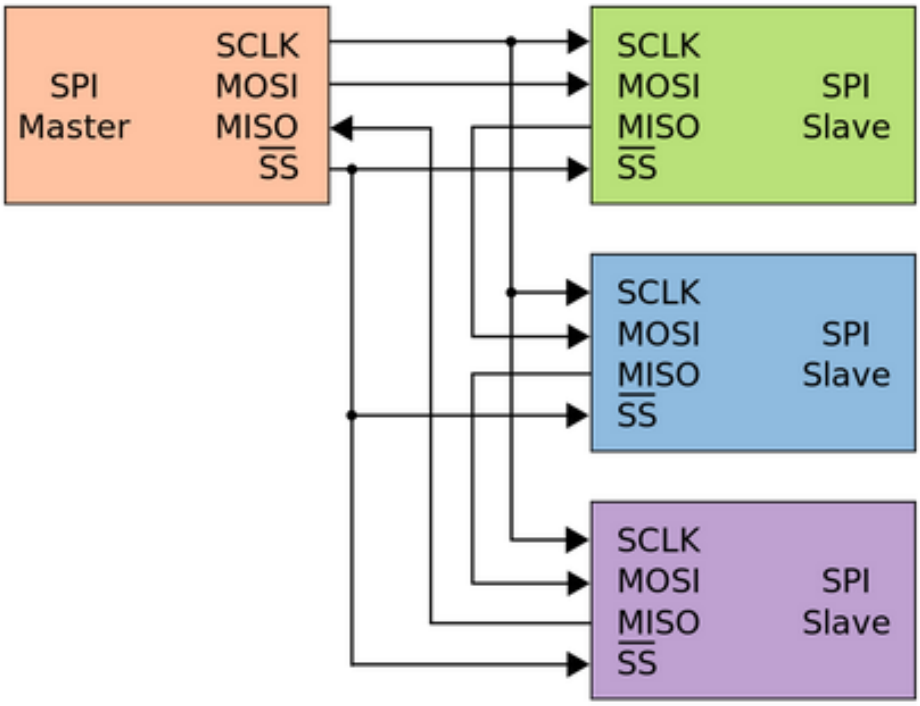
2. Conectarea mai multor dispozitive Slave

Mai multe dispozitive Slave pot fi conectate în același timp la un singur dispozitiv Master. Semnalele MOSI, MISO și SCLK sunt partajate. Pentru fiecare dispozitiv Slave, dispozitivul Master are câte un semnal CS / SS. Când Masterul dorește să comunice cu un Slave, dispozitivul Master setează pe LOW semnalul CS / SS care duce către dispozitivul Slave dorit (celelalte semnale CS / SS sunt puse pe HIGH). Astfel, dispozitivul Slave cu CS / SS pe LOW știe că Masterul comunică cu el.



3. Conectarea mai multor dispozitive Slave cu ajutorul topologiei SPI Daisy Chain

În topologia Daisy Chain, datele sunt transferate de la Master la primul Slave, de la primul Slave la al doilea Slave, până când datele ajung la ultimul Slave care le trimite înapoi la Master. Datele primului Slave ajung la Master ultimele, în timp ce datele ultimului Slave ajung la Master primele.

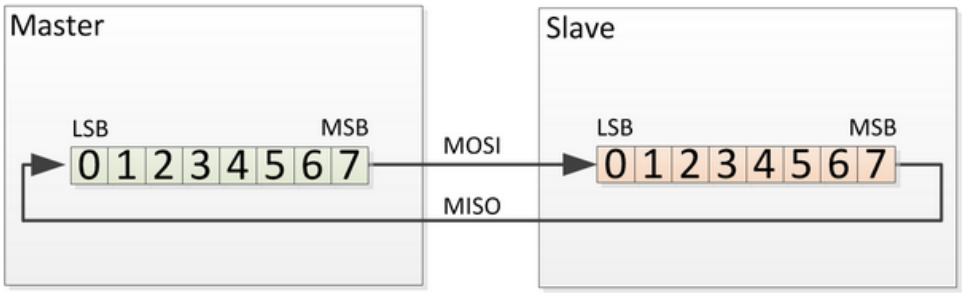


4. Transmiterea datelor cu SPI

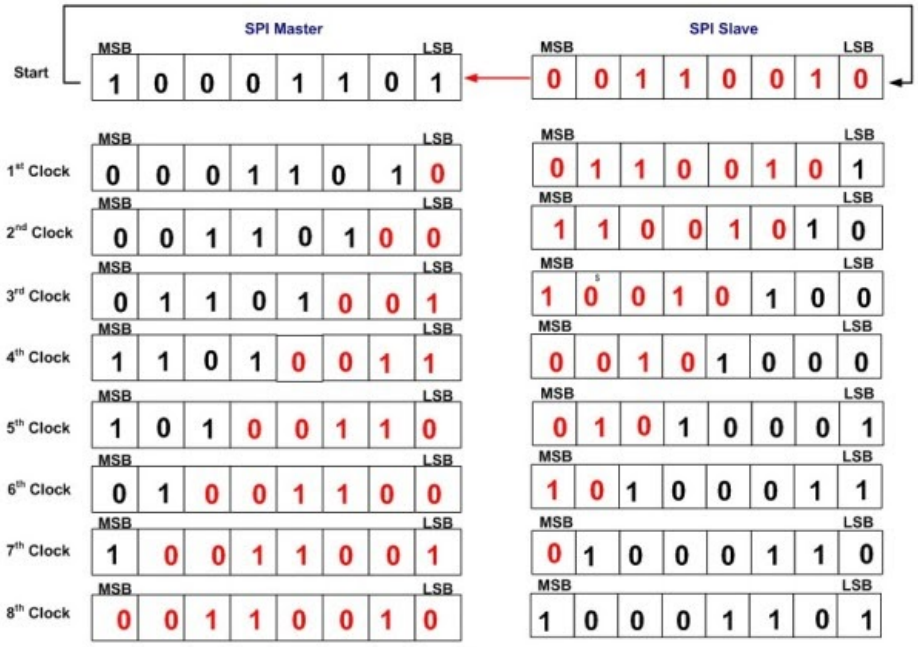
Pentru a începe comunicarea cu dispozitivul Master, ceasul Master-ului trebuie setat la o frecvență cel mult egală cu frecvența suportată de dispozitivul Slave (de obicei până la câțiva MHz). Apoi, Master-ul selectează dispozitivul Slave dorit, punând 0 pe linia CS / SS. În timpul unui ciclu SPI, transmisia este full-duplex:

- Dispozitivul Master trimite un bit pe linia MOSI, iar dispozitivul Slave citește acest bit.
- Dispozitivul Slave trimite un bit pe linia MISO, iar dispozitivul Master citește acest bit.

De obicei, comunicarea SPI implică două registre de șiftare (unul în Master și unul în Slave, conectate circular).



De obicei, primul bit mutat pe MISO / MOSI este cel mai semnificativ bit, în timp ce un bit nou este adăugat la poziția cea mai puțin semnificativă din registru. După ce întregul cuvânt a fost trimis, prin șiftare, Master-ul și Slave-ul au schimbat integral valorile din cele două registre de șiftare. Dacă mai există date de transmis, procesul începe din nou. Când nu există date de transmis, dispozitivul Master întrerupe generarea semnalului de ceas (SCLK) și plasează semnalul CS / SS asociată cu Slave-ul pe HIGH (1 logic). Dispozitivele Slave care nu au fost selectate de semnalul SS asociat vor ignora semnalele de pe SCLK și MOSI și nu vor genera nimic pe MISO. Master-ul poate selecta un singur Slave la un moment dat de timp.



SPI Master and Slave Data Transfer Diagram

5. Configurări de SPI

Clock Polarity (CPOL) configurează starea IDLE a ceasului. După cum putem vedea, pentru CPOL = 0, când ceasul este inactiv valoarea semnalului SCLK este pe LOW, iar pentru CPOL = 1 ceasul inactiv are SCLK pe HIGH.

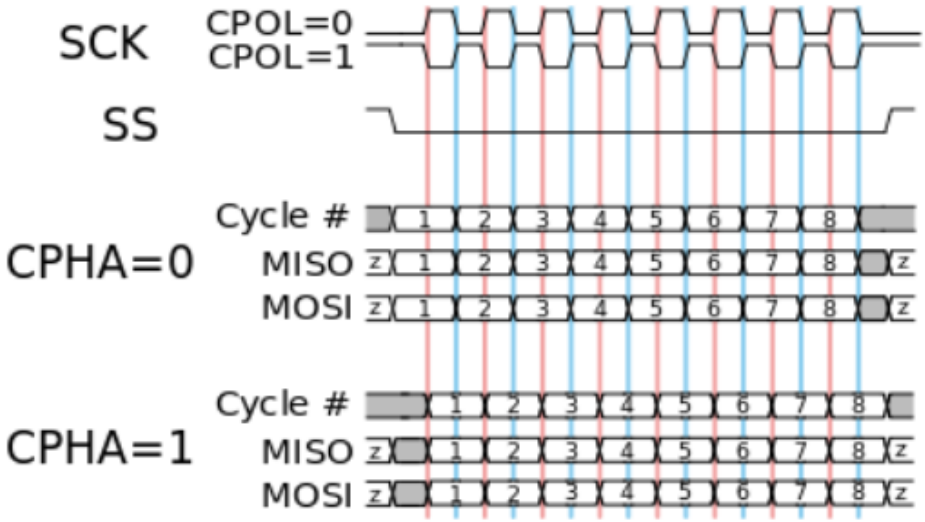
Clock Phase (CPHA) configurează momentul când datele sunt generate pe ieșire și când datele sunt citite la intrare. Pe unul dintre fronturile ceasului (de întâmpinare sau de sfârșit) datele sunt generate, iar pe cealaltă datele sunt citite.

Frontul de pornire al ceasului este prima schimbare de ceas. Ex: Pentru un ceas care merge de la LOW la HIGH și înapoi la LOW, frontul de întâmpinare este frontul crescător(tranziția de la LOW la HIGH).

Pentru CPHA = 0 datele sunt generate înainte de frontul de întâmpinare (primul front al ceasului) și sunt citite pe pe frontul întâmpinare (frontul crescător pentru CPOL = 0 și descrescător pentru CPOL = 1).

Pentru CPHA = 1 datele sunt generate pe frontul de întâmpinare (primul front al ceasului) și sunt citite pe frontul de sfârșit (frontul descrescător pentru CPOL = 0 și crescător pentru CPOL = 1).

CPOL și CPHA trebuie setate în conformitate cu configurația dispozitivului SPI cu care comunicăm. De exemplu, dacă UC-ul nostru comunică cu un ADC care utilizează CPOL = 1 și CPHA = 1, este obligatoriu dispozitivul nostru să aibă aceleași configurări de SPI.



6. SPI în Atmega328p

SPI-ul inclus în microcontrolerul atmega328p poate funcționa atât ca Master, cât și ca Slave.

SPI Control Register (SPCR)

Bit	7	6	5	4	3	2	1	0	
0x2C (0x4C)	SPIE	SPE	DORD	MSTR	CPOL	CPHA	SPR1	SPR0	SPCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Bit 6 - SPE (SPI Enable)
  - 0 - SPI dezactivat
  - 1 - SPI activat
- Bit 5 - DORD (Data Order)
  - 0 - MSB (din data) trimis primul
  - 1 - LSB (din data) trimis primul
- Bit 4 - MSTR (Master/Slave Select)
  - 0 - Slave mode
  - 1 - Master mode

- Bit 3 - CPOL (Clock Polarity)
  - 0 - SCK LOW pe idle (când nu comunică)
  - 1 - SCK HIGH pe idle (când nu comunică)

- Bit 2 - CPHA (Clock Phase)
  - 0 - datele sunt citite pe frontul de întâmpinare
  - 1 - datele sunt citite pe frontul de sfârșit

- Bit 1:0 - SPR1:SPR0 (SPI Clock Rate)
  - în modul Master: controlează frecvența SCK-ului.
  - În modul Slave: nu au efect.

SPI2X	SPR1	SPR0	SCK Frequency
0	0	0	$f_{osc}/4$
0	0	1	$f_{osc}/16$
0	1	0	$f_{osc}/64$
0	1	1	$f_{osc}/128$
1	0	0	$f_{osc}/2$
1	0	1	$f_{osc}/8$
1	1	0	$f_{osc}/32$
1	1	1	$f_{osc}/64$

SPI Status Register (SPSR)

Bit	7	6	5	4	3	2	1	0	
0x2D (0x4D)	SPIF	WCOL	–	–	–	–	–	SPI2X	SPSR
Read/Write	R	R	R	R	R	R	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Bit 7 – SPIF (SPI Interrupt Flag)
  - este automat setat pe 1 când transmisia s-a terminat.

- Bit 0 - SPI2X (Double SPI Speed Bit)
  - în modul Master: când acest bit este setat la 1, viteza SPI (Frecvența SCK) va fi dublată. Aceasta înseamnă că perioada minimă SCK va fi de două perioade de ceas de CPU.
  - în modul Slave:SPI este doar garantat că o să funcționeze la fosc / 4 sau mai mic.

SPI Data Register (SPDR)

Bit	7	6	5	4	3	2	1	0	
0x2E (0x4E)	MSB							LSB	SPDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	X	X	X	X	X	X	X	X	Undefined

Un registru de citire / scriere utilizat pentru transferul de date între Register File și SPI Shift Register. Scrierea în registru inițiază transmiterea datelor. Citirea registrului determină citirea bufferului Shift Register Receive.

Exemplu

Inițializare SPI ca Master.

```
/* set master mode */
SPCR |= (1 << MSTR);

/* set prescaler 16 */
SPCR |=  (1 << SPR);

/* enable SPI  */
SPCR |= (1 << SPE);
```

Transmiterea unui byte (pe Master).

```
/* Start transmission */
SPDR = data;

/* Wait for transmission complete */
while(!(SPSR & (1 << SPIF)));
```

Recepția unui byte (pe Slave).

```
/* Wait for reception complete */
while(!(SPSR & (1 << SPIF)));

/* Return Data Register */
return SPDR;
```

7. SPI în Arduino IDE

Mediul de dezvoltare Arduino IDE ne oferă posibilitatea de a folosi biblioteca "SPI.h" pentru a facilita comunicarea prin SPI. Mai jos puteți observa un exemplu de cod pentru transmiterea unui byte de date:

- SPI.h

```
/* Exemplu SPI MASTER: transfer un byte către Slave */
#include "SPI.h"

void setup(void)
{
  byte masterSend = 1;
  byte masterRecv;

  /* Inițializarea pinului de Slave Select */
  pinMode(SS, OUTPUT);
  digitalWrite(SS, HIGH);

  /* Inițializarea SPI-ului */
  SPI.begin();

  /* Selectarea Slave-ului */
  digitalWrite(SS, LOW);

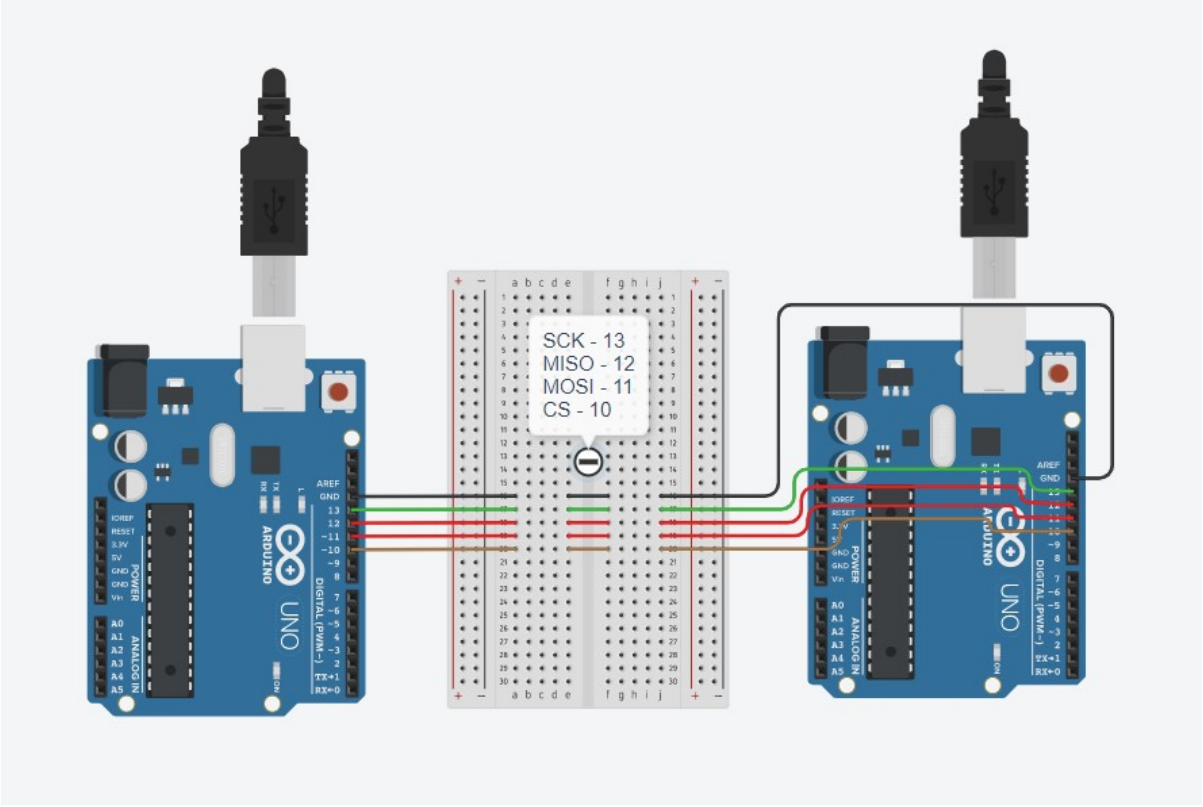
  /* Trimiterea byte-ului masterSend către Slave-ul selectat.
   * Salvarea byte-ului primit de la Slave-ul selectat în masterRecv. */
  masterRecv = SPI.transfer(masterSend);

  /* Deselectarea Slave-ului */
  digitalWrite(SS, HIGH);
}
```

8. Exerciții

1. Realizați o comunicație prin SPI între 2 plăcuțe Arduino.
    - Hotărâți ce plăcuță va fi Master și care va fi Slave (Câte o echipă poate alege să facă codul pentru Master și cealaltă pentru Slave).

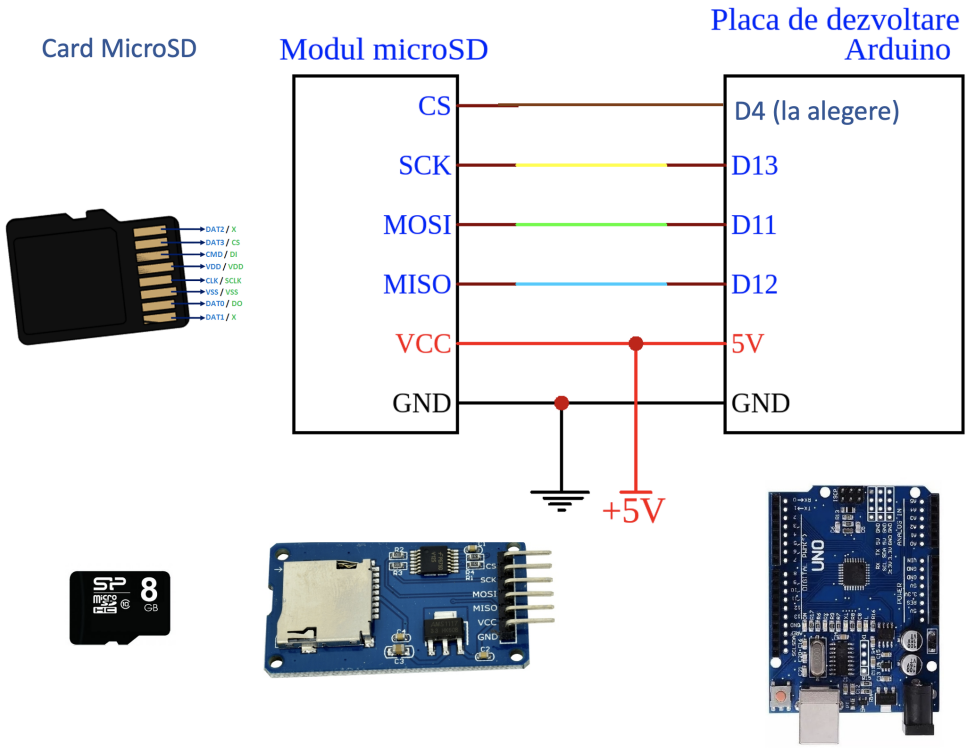
- Trimiteți un mesaj de la Master către Slave, cât și de la Slave la Master și afișați-le pe interfața serială. Stabiliți o lungime minimă a informației ce trebuie trimisă înainte să opriți transmiterea datelor.
- Pentru a putea să detectați comunicația SPI din Slave puteți implementa întreruperea SPI_STC_vect (trebuie să adaugați în setup "SPI.attachInterrupt();" - cum trebuiau să fie declarate variabilele modificate în întreruperi?). După ce ați încărcat codul pe plăcuțe, resetați-le pe ambele (apăsând butonul roșu de pe acestea), întâi plăcuța Slave (care va aștepta apoi inițierea comunicației) și apoi plăcuța Master (care va inița comunicația).



1. **(Bonus)** Un card MicroSD este un exemplu de memorie nevolatilă. Pentru ca plăcuța Arduino UNO să poată comunica prin SPI cu cardul, avem nevoie de un adaptor. În privința codului, pe lângă biblioteca "SPI.h" prezentată mai sus avem nevoie și de biblioteca "SD.h" despre care vă sfătuim să citiți înainte să continuați cu exercițiile.
    - a) Scrieți un secret într-un fișier numit "sd.txt" pe cardul SD.
    - b) Faceți schimb de carduri cu colegii voștri și citiți-le secretul.
    - c) Realizați un contor al resetărilor plăcuței Arduino scriind pe cardul SD în fișierul "re.txt" numărul de resetări.

Hint1: La resetare se rulează din nou codul din setup.
Hint2: Salvați pe cardul SD ultima valoare a numărului de resetări.

Dacă ați modificat montajul pentru a realiza exercițiul bonus, refaceți schema exercițiului 1 înainte de a pleca!



- SD.h

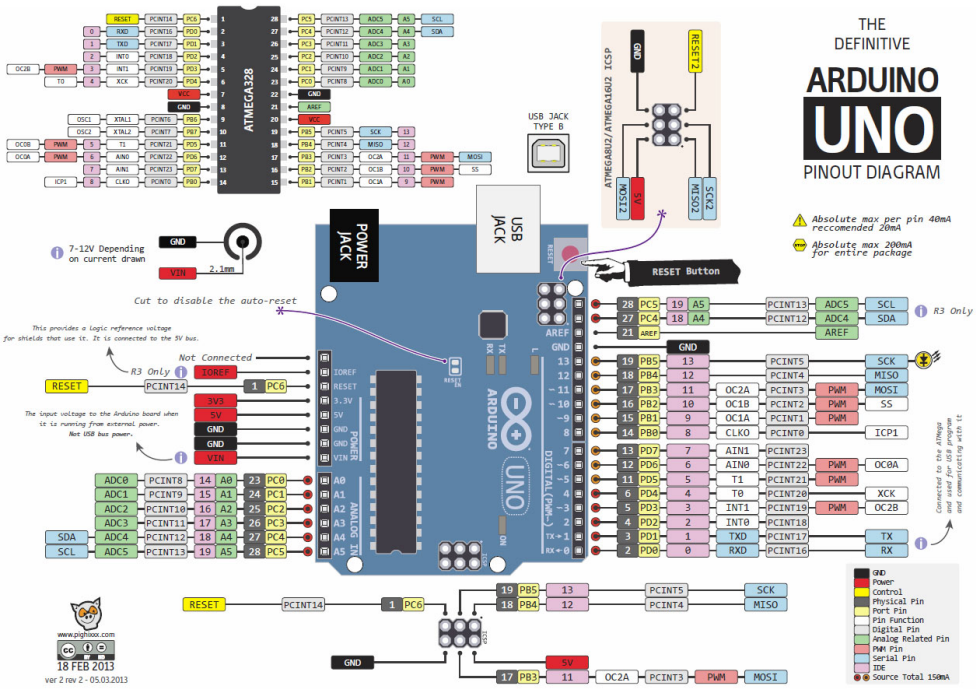
```
/* Exemplu MASTER: Creare și scriere a unui fișier pe Card MicroSD */
#include "SD.h"

void setup(void)
{
  File myFile;

  /* Inițializare comunicare cu Cardul MicroSD. Semnalul SS este pe pinul 4 */
  if (SD.begin(4))
  {
    /* Creăm fișierul dacă acesta nu există */
    if (!SD.exists("hello.txt"))
    {
      myFile = SD.open("hello.txt", FILE_WRITE);
      if (myFile)
      {
        myFile.print("Hello world");
        myFile.close();
      }
      else
      {
        /* Fișierul nu a putut fi creat. Trebuie investigată problema. */
      }
    }
  }
  else
  {
    /* Inițializarea comunicării a eșuat. Trebuie investigată problema. */
  }
}
```

9. Resurse

- Datasheet Atmega 328p
- Arduino UNO pinout



- Responsabili: Bogdan-Călin Ciobanu [mailto:bogdan.ciobanu1201@stud.acs.upb.ro] Ionuț-Gabriel Oțelea [mailto:ionut.otelea@upb.ro]

Rezolvare

10. Linkuri Utile

- Arduino SPI [https://www.arduino.cc/en/reference/SPI]
- Arduino SD [https://www.arduino.cc/en/reference/SD]
- Datasheet ATmega328p [https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf]

Laboratorul 6: I2C (Inter-Integrated Circuit)

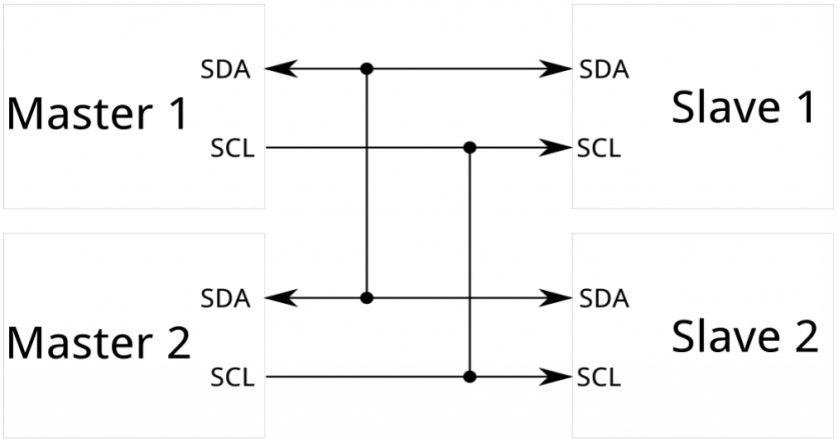
Acest laborator acoperă noțiunea de I2C. Pentru aprofundarea acestui topic, consultați Datasheet ATmega328P [https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf] și Inter-Integrated Circuit.

1. I2C (Inter-Integrated Circuit)

Protocolul I2C (sau IIC - Inter-Integrated Circuit / TWI - Two-Wire Interface) este un protocol de comunicație serială sincron, multi-master - multi-slave, dezvoltat de către Phillips în anul 1982. O magistrală I2C utilizează următoarele semnale:

- SDA - linia de date
- SCL - semnalul de clock

Semnalul de ceas este generat de către master, însă linia de date este controlată ori de către master, ori de către slave, în funcție de faza protocolului, cu restricția ca, la un moment dat, un singur dispozitiv să transmită. Din acestă cauză protocolul I2C este half-duplex.



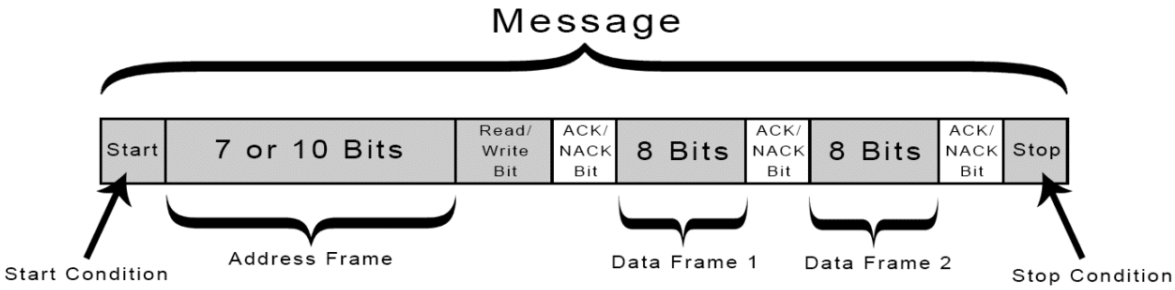
I2C mai este cunoscut și cu numele de TWI (Two-Wire Interface)

2. Modul de funcționare

Față de SPI unde master-ul activa, prin intermediul semnalului de *Slave Select*, dispozitivul cu care dorea să comunice, I2C nu necesită un asemenea semnal adițional. Protocolul I2C introduce noțiunea de *Slave Address*. Adresa unui dispozitiv de tip slave este un număr pe 7 biți (cel mai comun), pe 8 biți sau 10 biți. Comunicația dintre un master și un slave se face prin mesaje și este tot timpul inițiată de către master. Aceste mesaje pot fi sparte în două tipuri de cadre:

- un cadru de adresă
- unul sau mai multe cadre de date

Aceste cadre sunt interschimbate numai după ce master-ul a trimis *condiția de start*. Sfârșitul unui mesaj este identificat prin *condiția de stop*.



1. Condiția de start

Înainte ca master-ul să trimită pe linia de date adresa slave-ului cu care dorește să comunice, acesta trebuie sa genereze o condiție de start. Condiția de start determină toate dispozitivele slave să ”asculte” linia de date pentru că va urma o adresă. Pentru a genera această condiție, master-ul lasă linia SCL in HIGH și pune linia SDA pe LOW.

2. Cadrul de adresă

După ce masterul a generat condiția de start, acesta trimite pe linia de date (SDA) adresa dispozitivului slave cu care dorește să comunice. Adresa este (de cele mai multe ori) un număr pe 7 biți (biții A6-A0). Bitul 0 menționează dacă master-ul inițiază o operație de Citire (bitul 0 este 1) sau o operație de Scriere (bitul 0 este 0).

Slave-ul care își recunoaște adresa trimite un ACK master-ului prin punerea liniei SDA pe LOW în al nouălea ciclu de ceas. Starea default a liniilor SDA/SCL este HIGH datorită rezistențelor de pull-up. Master-ul/Slave-ul doar ”trag” liniile pe LOW.

Master-ul identifică dacă a primit ACK (SDA pus pe LOW) sau NACK (SDA a rămas HIGH pe durata celui de-al nouălea ciclu de ceas).

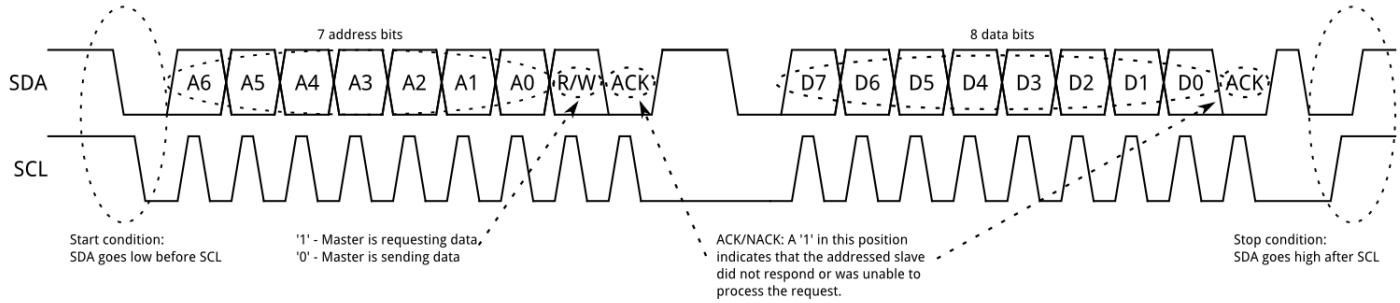
3. Cadrele de date

Dacă master-ul a primit ACK (dacă există un slave pe magistrală cu adresa respectivă), el poate continua cu transmiterea datelor (operație de scriere), sau cu recepția datelor (operație de citire). Numărul de cadre de date este arbitrar, pot fi interschimbate oricâte. Fiecare cadru trimis/recepționat este ACK'd sau NACK'd. În funcție de operație (citire sau scriere), ACK-ul/NACK-ul este trimis fie de master fie de slave.

- Daca master-ul a inițiat o operație de scriere, fiecare cadru trimis este confirmat (ACK'd) de către slave.
- Daca master-ul a inițiat o operatie de citire, fiecare cadru recepționat este confirmat de (ACK'd) de master. Când master-ul dorește să oprească tranzacția după ce un anumit număr de cadre a fost recepționat, în loc să trimită ACK trimite NACK. Astfel slave-ul se va opri din transmitere.

4. Condiția de stop

După ce toate cadrele de date au fost interschimbate, master-ul generează condiția de stop. Aceasta este realizată prin eliberarea liniei SDA (trecere din LOW în HIGH) **după** eliberarea liniei SCL (trecere din LOW în HIGH).



Magistrala este aceeași pentru toți senzorii iar fiecare slave este referit printr-o adresa. Astfel, dacă avem mai mulți slave conectați pe magistrala, masterul alege cu care dintre ei dorește sa initieze comunicatia, precizand adresa specificata în datasheetul senzorului. Cel mai adesea, se folosesc adrese reprezentate pe 7 biti, iar cel mai nesemnificativ bit din octetul adresei ne arata daca urmeaza o operatie de scriere sau citire. După identificarea dispozitivului cu care se comunică, se poate începe transmiterea datelor.

3. Registre configurare I2C

Arduino poate funcționa atât în modul I2C Master cât și I2C Slave.

TWI Bit Rate Register

Bit	7	6	5	4	3	2	1	0	
(0xB8)	TWBR7	TWBR6	TWBR5	TWBR4	TWBR3	TWBR2	TWBR1	TWBR0	TWBR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- TWBR selectează divizorul de frecvență care generează frecvența ceasului SCL în modurile master.

TWI Control Register

Bit	7	6	5	4	3	2	1	0	
(0xBC)	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	–	TWIE	TWCR
Read/Write	R/W	R/W	R/W	R/W	R	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- TWCR este folosit pentru a activa interfața I2C, inițiază accesul la master prin aplicarea condiției START, generează confirmarea receptorului, generează condiția de oprire și control al acesteia în timpul scrierii în registrul de date TWDR. De asemenea, indică coliziuni la scriere.
- TWINT : TWI Interrupt Flag
  - 1 - Clear, setat din software, in rest este setat de hardware.
- TWEA: TWI Enable Acknowledge Bit
  - 1 - ACK este trimis
  - 0 - NACK, deconectare virtuală a dispozitivului de pe interfață
- TWEN: TWI Enable Bit
  - 1 - Permite comunicarea
  - 0 - Încheierea transmisiunii indiferent de stare

TWI Status Register

Bit	7	6	5	4	3	2	1	0	
(0xB9)	TWS7	TWS6	TWS5	TWS4	TWS3	–	TWPS1	TWPS0	TWSR
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial Value	1	1	1	1	1	0	0	0	

- TWSR descrie statusul si prescalerul interfeței.

TWI Data Register

Bit	7	6	5	4	3	2	1	0	
(0xBB)	TWD7	TWD6	TWD5	TWD4	TWD3	TWD2	TWD1	TWD0	TWDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	1	1	1	1	1	1	1	1	

- TWDR conține în modul de transmisie următorul octet de transmis iar în modul de recepție ultimul octet primit.

4. Utilizarea modulului I2C pe AVR

Procesul de configurare al modulului I2C pe microcontrollerele AVR este relativ simplu (trebuie setată doar frecvența ceasului):

I2C Setup:

```
void setup()
{
  // Initialize I2C to a 100KHz clock
  // TWI Status Register: initialize prescaler to 1
  TWSR = (0b00 << TWPS0);
  // TWI Bitrate Register: set bitrate
  // SCL_Freq = CPU_Freq / (16 + 2*TWBR * TWSR_Prescaler)
  // so: TWBR = (SCL_Freq / CPU_Freq - 16) / (TWSR_Prescaler * 2)
  TWBR = 72; // (16000000/100000 - 16) / (1 * 2)
}
```

Însă pașii pentru a efectua o tranzacție completă pe I2C sunt complecși:

- configurare TWCR pentru a transmite condiția de start;
- așteptarea completării operațiunii (prin busy waiting pe TWSR);
- scrierea adresei dispozitivului slave în registrul TWDR + așteptarea;
- verificarea condiției de acknowledgement (tot prin TWSR);
- transmiterea sau primirea a zero sau mai mulți bytes prin intermediul registrului de date TWDR (de asemenea, trebuie incluși pașii intermediari de așteptare a finalizării operațiilor);
- în final, transmiterea condiție de stop printr-un alt flag din TWCR.

5. Utilizarea TWI prin biblioteca Arduino

Desigur, Arduino dispune de funcții de nivel înalt pentru operațiunile pe magistrala I2C a microcontrollerului, exemplu:

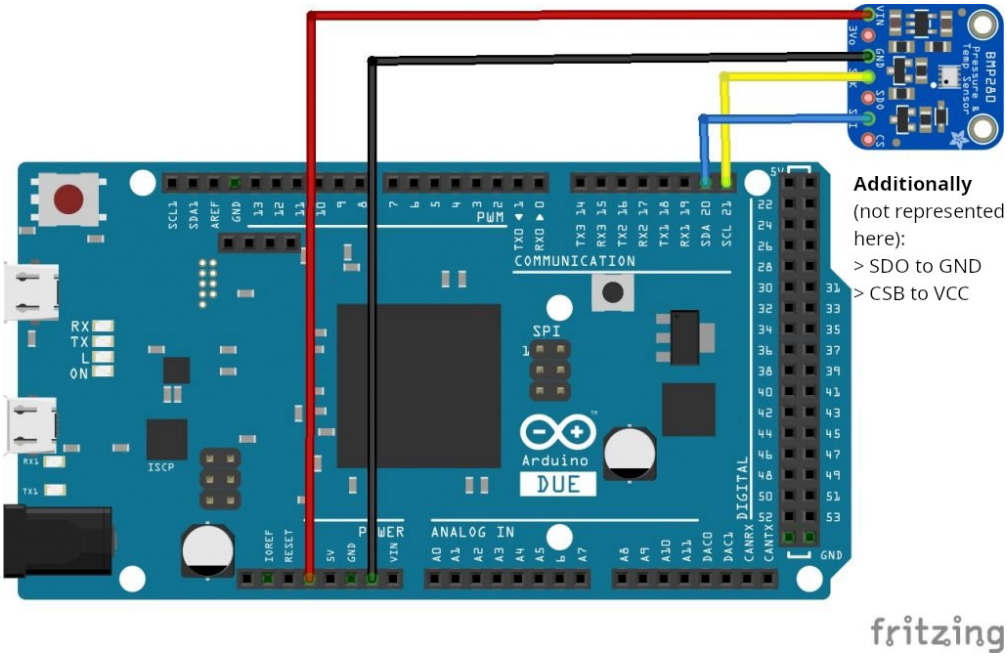
```
void setup()
{
  Wire.setClock(100000); // Hz
  Wire.begin();
}
```

```
void loop()
{
  Wire.beginTransmission(0x52);  // slave address
  Wire.write(0x88);
  Wire.endTransmission();
  delay(5000);
}
```

We don't do that here!

6. Exerciții

Pentru laboratorul acesta, vom utiliza un Arduino Uno cu rolul de master, la care a fost conectat un senzor de temperatură + presiune **BMP280** (click pentru datasheet [https://cdn-shop.adafruit.com/datasheets/BST-BMP280-DS001-11.pdf]) de la care vom citi datele.



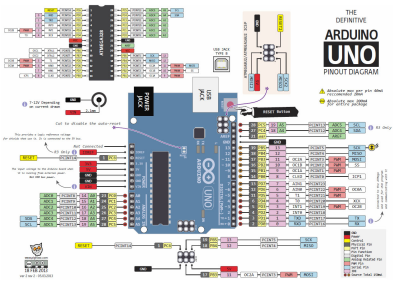
Pentru acesta, va trebui să descărcați scheletul de cod aflat aici: lab06-i2c.zip.

Task 1: Definitivați și testați codul pentru a realiza scanarea liniară a dispozitivelor I2C conectate. Ar trebui să găsiți dispozitivul la adresa `0x76`.

Task 2: Definitivați funcțiile de scriere / citire regiștrii ai dispozitivului și folosiți-le pentru a citi + converti temperatura în grade Celsius.

7. Resurse

- Datasheet Atmega 328p
- Schelet de cod Lab 6 I2C
- Arduino UNO pinout



- Responsabil: Florin Stancu [mailto:florin.stancu@upb.ro], Daniel Rosner [mailto:daniel.rosner@upb.ro], Cristi Trancă [mailto:dumitru.tranca@upb.ro]

Rezolvare

8. Linkuri Utile

- Arduino I2C [https://www.arduino.cc/en/Reference/Wire]
- Datasheet ATmega328p [https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf]
- Datasheet Senzor BMP280 [https://cdn-shop.adafruit.com/datasheets/BST-BMP280-DS001-11.pdf]

pm/lab/lab6-2022.txt · Last modified: 2023/03/05 15:02 by dragos_mihai.ene